

# An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation\*

Matthew Kelly<sup>†</sup>

**Abstract.** This paper is an introductory tutorial for numerical trajectory optimization with a focus on direct collocation methods. These methods are relatively simple to understand and effectively solve a wide variety of trajectory optimization problems. Throughout the paper we illustrate each new set of concepts by working through a sequence of four example problems. We start by using trapezoidal collocation to solve a simple one-dimensional toy problem and work up to using Hermite–Simpson collocation to compute the optimal gait for a bipedal walking robot. Along the way, we cover basic debugging strategies and guidelines for posing well-behaved optimization problems. The paper concludes with a short overview of other methods for trajectory optimization. We also provide an electronic supplement that contains well-documented MATLAB code for all examples and methods presented. Our primary goal is to provide the reader with the resources necessary to understand and successfully implement their own direct collocation methods.

**Key words.** trajectory optimization, optimal control, direct collocation, tutorial, direct transcription, robotics

**AMS subject classifications.** 34, 37, 49, 90, 97

**DOI.** 10.1137/16M1062569

## Contents

|          |                                                       |            |
|----------|-------------------------------------------------------|------------|
| <b>I</b> | <b>Introduction</b>                                   | <b>851</b> |
| 1.1      | Overview . . . . .                                    | 852        |
| 1.2      | Notation . . . . .                                    | 852        |
| 1.3      | A Simple Example . . . . .                            | 852        |
| 1.4      | The Trajectory Optimization Problem . . . . .         | 853        |
| 1.5      | Direct Collocation Method . . . . .                   | 854        |
| 1.6      | Nonlinear Programming . . . . .                       | 855        |
| <b>2</b> | <b>Block-Move Example (Minimum-Force Objective)</b>   | <b>855</b> |
| 2.1      | Block-Move Example: Problem Statement . . . . .       | 855        |
| 2.2      | Block-Move Example: Analytic Solution . . . . .       | 856        |
| 2.3      | Block-Move Example: Trapezoidal Collocation . . . . . | 856        |
| 2.4      | Initialization . . . . .                              | 857        |
| 2.5      | Block-Move Example: Nonlinear Program . . . . .       | 857        |

\*Received by the editors February 22, 2016; accepted for publication (in revised form) December 22, 2016; published electronically November 6, 2017.

<http://www.siam.org/journals/sirev/59-4/M106256.html>

**Funding:** This work was supported by the National Science Foundation.

<sup>†</sup>Department of Mechanical Engineering, Cornell University, Ithaca, NY 14850 (mpk72@cornell.edu).

|          |                                                              |            |
|----------|--------------------------------------------------------------|------------|
| 2.6      | Block-Move Example: Interpolation . . . . .                  | 858        |
| <b>3</b> | <b>Trapezoidal Collocation Method</b>                        | <b>858</b> |
| 3.1      | Trapezoidal Collocation: Integrals . . . . .                 | 858        |
| 3.2      | Trapezoidal Collocation: System Dynamics . . . . .           | 858        |
| 3.3      | Trapezoidal Collocation: Constraints . . . . .               | 858        |
| 3.4      | Trapezoidal Collocation: Interpolation . . . . .             | 859        |
| <b>4</b> | <b>Hermite–Simpson Collocation Method</b>                    | <b>860</b> |
| 4.1      | Hermite–Simpson Collocation: Integrals . . . . .             | 861        |
| 4.2      | Hermite–Simpson Collocation: System Dynamics . . . . .       | 861        |
| 4.3      | Hermite–Simpson Collocation: Constraints . . . . .           | 862        |
| 4.4      | Hermite–Simpson Collocation: Interpolation . . . . .         | 862        |
| <b>5</b> | <b>Practical Considerations</b>                              | <b>863</b> |
| 5.1      | Initialization . . . . .                                     | 863        |
| 5.2      | Mesh Refinement . . . . .                                    | 864        |
| 5.3      | Error Analysis . . . . .                                     | 865        |
| 5.4      | Debugging Your Code . . . . .                                | 866        |
| 5.5      | Consistent Functions . . . . .                               | 867        |
| <b>6</b> | <b>Cart-Pole Swing-Up Example</b>                            | <b>868</b> |
| 6.1      | Cart-Pole Example: System Dynamics . . . . .                 | 868        |
| 6.2      | Cart-Pole Example: Objective Function . . . . .              | 869        |
| 6.3      | Cart-Pole Example: Boundary Constraints . . . . .            | 869        |
| 6.4      | Cart-Pole Example: State and Control Bounds . . . . .        | 870        |
| 6.5      | Cart-Pole Example: Trapezoidal Collocation . . . . .         | 870        |
| 6.6      | Cart-Pole Example: Hermite–Simpson Collocation . . . . .     | 870        |
| 6.7      | Cart-Pole Example: Initialization . . . . .                  | 871        |
| 6.8      | Cart-Pole Example: Results . . . . .                         | 871        |
| <b>7</b> | <b>Five-Link Biped Example</b>                               | <b>873</b> |
| 7.1      | Five-Link Biped: Model . . . . .                             | 873        |
| 7.2      | Five-Link Biped: System Dynamics . . . . .                   | 874        |
| 7.3      | Five-Link Biped: Objective Function . . . . .                | 874        |
| 7.4      | Five-Link Biped: Constraints . . . . .                       | 875        |
| 7.5      | Five-Link Biped: Initialization . . . . .                    | 876        |
| 7.6      | Five-Link Biped: Results . . . . .                           | 876        |
| <b>8</b> | <b>Block-Move Example (Minimum-Work)</b>                     | <b>877</b> |
| 8.1      | Block-Move Example: Problem Statement . . . . .              | 878        |
| 8.2      | Block-Move Example: Analytic Solution . . . . .              | 879        |
| 8.3      | Block-Move Example: Discontinuities . . . . .                | 880        |
| 8.4      | Block-Move Example: Initialization . . . . .                 | 880        |
| 8.5      | Block-Move Example: Slack Variables . . . . .                | 880        |
| 8.6      | Block-Move Example: Smoothing . . . . .                      | 881        |
| 8.7      | Block-Move Example: Results . . . . .                        | 882        |
| <b>9</b> | <b>Background</b>                                            | <b>883</b> |
| 9.1      | Trajectory Optimization vs. Parameter Optimization . . . . . | 883        |
| 9.2      | Open-Loop vs. Closed-Loop Solutions . . . . .                | 883        |

|                                                                  |            |
|------------------------------------------------------------------|------------|
| AN INTRODUCTION TO TRAJECTORY OPTIMIZATION                       | 851        |
| 9.3 Continuous-Time and Discrete-Time Systems . . . . .          | 884        |
| 9.4 Indirect Methods . . . . .                                   | 884        |
| 9.5 Direct Single Shooting . . . . .                             | 885        |
| 9.6 Direct Multiple Shooting . . . . .                           | 885        |
| 9.7 Orthogonal Collocation . . . . .                             | 886        |
| 9.8 Differential Dynamic Programming . . . . .                   | 886        |
| 9.9 Multiphase Methods . . . . .                                 | 886        |
| 9.10 Through-Contact Methods . . . . .                           | 886        |
| 9.11 Which Method Is Best? . . . . .                             | 886        |
| 9.12 Trajectory Optimization Software . . . . .                  | 887        |
| <b>I0 Summary</b>                                                | <b>887</b> |
| <b>Appendix A. Overview of Electronic Supplementary Material</b> | <b>888</b> |
| A.1 Trajectory Optimization Code . . . . .                       | 888        |
| A.2 Example Problems . . . . .                                   | 889        |
| <b>Appendix B. Analytic Solution to Block-Move Example</b>       | <b>889</b> |
| B.1 Full Solution . . . . .                                      | 889        |
| B.2 Short Solution . . . . .                                     | 892        |
| <b>Appendix C. Derivation of Simpson Quadrature</b>              | <b>892</b> |
| <b>Appendix D. Orthogonal Polynomials</b>                        | <b>893</b> |
| D.1 Computing Polynomial Roots . . . . .                         | 894        |
| D.2 Barycentric Lagrange Interpolation . . . . .                 | 895        |
| D.3 Differentiation Matrix . . . . .                             | 895        |
| D.4 Quadrature . . . . .                                         | 896        |
| <b>Appendix E. Parameters for Example Problems</b>               | <b>896</b> |
| E.1 Cart-Pole Swing-Up Parameters . . . . .                      | 896        |
| E.2 Five-Link Biped Parameters . . . . .                         | 896        |
| <b>Appendix F. Biped Dynamics</b>                                | <b>897</b> |
| F.1 Kinematics . . . . .                                         | 897        |
| F.2 Single-Stance Dynamics . . . . .                             | 898        |
| F.3 Heel-Strike Dynamics . . . . .                               | 899        |
| F.4 Gradients . . . . .                                          | 901        |
| <b>References</b>                                                | <b>901</b> |

**I. Introduction.** What is trajectory optimization? Let's start with an example: imagine a satellite moving between two planets. We would use the term *trajectory* to describe the path the satellite takes between the two planets. Usually, this path would include both state (e.g., position and velocity) and control (e.g., thrust) as functions of time. The term *trajectory optimization* refers to a set of methods that are used to find the best choice of trajectory, typically by selecting the inputs to the system, known as *controls*, as functions of time.

**1.1. Overview.** Why read this paper? Our contribution is to provide a tutorial that covers all of the basics required to understand and implement direct collocation methods, while still being accessible to a broad audience. Where possible, we teach through examples, both in this paper and in the electronic supplement.

This tutorial starts with a brief introduction to the basics of trajectory optimization (section 1), and then it moves on to solve a simple example problem using trapezoidal collocation (section 2). The next sections cover the general implementation details for trapezoidal collocation (section 3) and Hermite–Simpson collocation (section 4), followed by a section about practical implementation details and debugging (section 5). Next there are three example problems: cart-pole swing-up (section 6), five-link bipedal walking (section 7), and minimum-work block-move (section 8). The paper concludes with an overview of related optimization topics and a summary of commonly used software packages (section 9).

This paper comes with a two-part electronic supplement, which is described in detail in Appendix A. The first part is a general purpose trajectory optimization library, written in MATLAB, that implements trapezoidal direct collocation, Hermite–Simpson direct collocation, direct multiple shooting (fourth-order Runge–Kutta), and global orthogonal collocation (Chebyshev Lobatto). The second part of the supplement is the full set of example problems from this paper implemented in MATLAB and solved using the aforementioned trajectory optimization library. The code in the supplement is well documented and designed to be read in a tutorial fashion.

**1.2. Notation.** For reference, the main symbols we will use throughout the tutorial and which are described in detail later are as follows:

|                                                               |                                                   |
|---------------------------------------------------------------|---------------------------------------------------|
| $t_k$                                                         | time at knot point $k$                            |
| $N$                                                           | number of trajectory (spline) segments            |
| $h_k = t_{k+1} - t_k$                                         | duration of spline segment $k$                    |
| $\mathbf{x}_k = \mathbf{x}(t_k)$                              | state at knot point $k$                           |
| $\mathbf{u}_k = \mathbf{u}(t_k)$                              | control at knot point $k$                         |
| $w_k = w(t_k, \mathbf{x}_k, \mathbf{u}_k)$                    | integrand of objective function at knot point $k$ |
| $\mathbf{f}_k = \mathbf{f}(t_k, \mathbf{x}_k, \mathbf{u}_k)$  | system dynamics at knot point $k$                 |
| $\dot{q} = \frac{d}{dt}q, \quad \ddot{q} = \frac{d^2}{dt^2}q$ | first and second time-derivatives of $q$          |

In some cases we will use the subscript  $k + \frac{1}{2}$  to indicate the midpoint of spline segment  $k$ . For example,  $\mathbf{u}_k$  gives the control at the beginning of segment  $k$ , and  $\mathbf{u}_{k+\frac{1}{2}}$  gives the control at the midpoint of segment  $k$ .

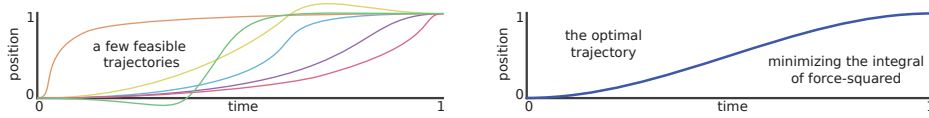
**1.3. A Simple Example.** We will start by looking at a simple example: how to move a small block between two points, starting and finishing at rest, in a fixed amount of time. First, we will need to write down the *dynamics*, which describe how the system moves. In this case, we will model the block as a point-mass that travels in one dimension, and the *control* (input) to the system is simply the force applied to the block. We use  $x$  for position,  $v$  for velocity, and  $u$  for control (force):

$$\dot{x} = v, \quad \dot{v} = u, \quad \text{system dynamics.}$$

We would like the block to move one unit of distance in one unit of time, and it should be stationary at both start and finish. These requirements are illustrated in Figure 1



**Fig. 1** Illustration of the boundary conditions for the simple block-move example.



**Fig. 2** Comparison of feasible (left) and optimal (right) trajectories for the simple block-move example.

and are known as *boundary conditions*:

$$\begin{aligned} x(0) &= 0, & x(1) &= 1, \\ \nu(0) &= 0, & \nu(1) &= 0, \end{aligned} \quad \text{boundary conditions.}$$

A solution to a trajectory optimization problem is said to be *feasible* if it satisfies all of the problem requirements, known as *constraints*. In general, there are many types of constraints. For the simple block-moving problem we have only two types of constraints: the system dynamics and the boundary conditions. Figure 2 shows several feasible trajectories. The set of controls that produce feasible trajectories are known as *admissible* controls.

Trajectory optimization is concerned with finding the best of the feasible trajectories, also known as the *optimal* trajectory, which is shown in Figure 2. We use an *objective function* to mathematically describe what we mean by the “best” trajectory. Later in this tutorial we will solve this block-moving problem with two commonly used objective functions: minimal force squared (section 2) and minimal absolute work (section 8):

$$\begin{aligned} \min_{u(t), x(t), \nu(t)} \int_0^1 u^2(\tau) d\tau, & \quad \text{minimum force squared,} \\ \min_{u(t), x(t), \nu(t)} \int_0^1 |u(\tau) \nu(\tau)| d\tau, & \quad \text{minimum absolute work.} \end{aligned}$$

**1.4. The Trajectory Optimization Problem.** There are many ways to formulate trajectory optimization problems [51, 5, 45]. Here we will restrict our focus to single-phase continuous-time trajectory optimization problems: those where the system dynamics are continuous throughout the entire trajectory. A more general framework is described in [51] and briefly discussed in section 9.9.

In general, an objective function can include two terms: a boundary objective  $J(\cdot)$  and a path integral along the entire trajectory, with the integrand  $w(\cdot)$ . A problem with both terms is said to be in *Bolza form*. A problem with only the integral term is said to be in *Lagrange form*, and a problem with only a boundary term is said to

be in *Mayer form* [5]. The examples in this paper are all in Lagrange form:

$$(1.1) \quad \min_{t_0, t_F, \mathbf{x}(t), \mathbf{u}(t)} \underbrace{J(t_0, t_F, \mathbf{x}(t_0), \mathbf{x}(t_F))}_{\text{Mayer Term}} + \underbrace{\int_{t_0}^{t_F} w(\tau, \mathbf{x}(\tau), \mathbf{u}(\tau)) d\tau}_{\text{Lagrange Term}}.$$

In optimization, we use the term *decision variable* to describe the variables that the optimization solver is adjusting to minimize the objective function. Generally, the decision variables are the initial and final time  $(t_0, t_F)$ , as well as the state and control trajectories,  $\mathbf{x}(t)$  and  $\mathbf{u}(t)$ , respectively.

The optimization is subject to a variety of limits and constraints, detailed in (1.2)–(1.9). The first, and perhaps most important, of these constraints is the system dynamics, which are typically nonlinear and describe how the system changes in time:

$$(1.2) \quad \dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t), \mathbf{u}(t)), \quad \text{system dynamics.}$$

Next is the path constraint, which enforces restrictions along the trajectory. A path constraint could be used, for example, to keep the foot of a walking robot above the ground during a step:

$$(1.3) \quad \mathbf{h}(t, \mathbf{x}(t), \mathbf{u}(t)) \leq \mathbf{0}, \quad \text{path constraint.}$$

Another important type of constraint is a nonlinear boundary constraint, which puts restrictions on the initial and final states of the system. Such a constraint would be used, for example, to ensure that the gait of a walking robot is periodic:

$$(1.4) \quad \mathbf{g}(t_0, t_F, \mathbf{x}(t_0), \mathbf{x}(t_F)) \leq \mathbf{0}, \quad \text{boundary constraint.}$$

Often there are constant limits on the state or control. For example, a robot arm might have limits on the angle, angular rate, and torque that could be applied throughout the entire trajectory:

$$(1.5) \quad \mathbf{x}_{\text{low}} \leq \mathbf{x}(t) \leq \mathbf{x}_{\text{upp}}, \quad \text{path bound on state,}$$

$$(1.6) \quad \mathbf{u}_{\text{low}} \leq \mathbf{u}(t) \leq \mathbf{u}_{\text{upp}}, \quad \text{path bound on control.}$$

Finally, it is often important to include specific limits on the initial and final time and state. These might be used to ensure that the solution to a path planning problem reaches the goal within some desired time window, or that it reaches some target region in state space:

$$(1.7) \quad t_{\text{low}} \leq t_0 < t_F \leq t_{\text{upp}}, \quad \text{bounds on initial and final time,}$$

$$(1.8) \quad \mathbf{x}_{0,\text{low}} \leq \mathbf{x}(t_0) \leq \mathbf{x}_{0,\text{upp}}, \quad \text{bound on initial state,}$$

$$(1.9) \quad \mathbf{x}_{F,\text{low}} \leq \mathbf{x}(t_F) \leq \mathbf{x}_{F,\text{upp}}, \quad \text{bound on final state.}$$

**1.5. Direct Collocation Method.** Most methods for solving trajectory optimization problems can be classified as either *direct* or *indirect*. In this tutorial we will focus on direct methods, although we do provide a brief overview of indirect methods in section 9.4. The key feature of a direct method is that it discretizes the trajectory optimization problem itself, typically converting the original trajectory optimization problem into a nonlinear program (see section 1.6). This conversion process is known as *transcription* and it is why some people refer to direct collocation methods as direct transcription methods.

In general, direct transcription methods are able to discretize a continuous trajectory optimization problem by approximating all of the continuous functions in the problem statement as polynomial splines. A *spline* is a function that is made up of a sequence of polynomial segments. Polynomials are used because they have two important properties: they can be represented by a small (finite) set of coefficients, and it is easy to compute integrals and derivatives of polynomials in terms of these coefficients.

Throughout this tutorial we will be studying two direct collocation methods in detail: trapezoidal collocation (section 3) and Hermite–Simpson collocation (section 4). We will also briefly cover a few other direct transcription techniques: direct single shooting (section 9.5), direct multiple shooting (section 9.6), and orthogonal collocation (section 9.7).

**1.6. Nonlinear Programming.** Most direct collocation methods transcribe a continuous-time trajectory optimization problem into a *nonlinear program*. A nonlinear program is a special name given to a constrained parameter optimization problem that has nonlinear terms in either its objective or its constraint function. A typical formulation for a nonlinear program is as follows:

$$(1.10) \quad \begin{aligned} \min_{\mathbf{z}} J(\mathbf{z}) \quad & \text{subject to} \\ & \mathbf{f}(\mathbf{z}) = \mathbf{0}, \\ & \mathbf{g}(\mathbf{z}) \leq \mathbf{0}, \\ & \mathbf{z}_{\text{low}} \leq \mathbf{z} \leq \mathbf{z}_{\text{upp}}. \end{aligned}$$

In this tutorial we will not spend time examining the details of how to solve a nonlinear program (see [35, 6, 11]), and instead we will focus on the practical details of how to properly use a nonlinear programming solver, such as those listed in section 9.12.

In some cases, a direct collocation method might produce either a linear or a quadratic program instead of a nonlinear program. This happens when the constraints (including system dynamics) are linear and the objective function is linear (linear program) or quadratic (quadratic program). Both linear and quadratic programs are much easier to solve than nonlinear programs, making them desirable for real-time applications, especially in robotics.

**2. Block-Move Example (Minimum-Force Objective).** In this section we continue with the simple example presented in the introduction: computing the optimal trajectory to move a block between two points.

**2.1. Block-Move Example: Problem Statement.** We will model the block as a unit point mass that slides without friction in one dimension. The *state* of the block is its position  $x$  and velocity  $\nu$ , and the *control* is the force  $u$  applied to the block:

$$(2.1) \quad \dot{x} = \nu, \quad \dot{\nu} = u.$$

Next, we need to write the *boundary constraints* which describe the initial and final states of the block. Here we constrain the block to move from  $x = 0$  at time  $t = 0$  to  $x = 1$  at time  $t = 1$ . Both the initial and final velocities are constrained to be zero:

$$(2.2) \quad \begin{aligned} x(0) &= 0, & x(1) &= 1, \\ \nu(0) &= 0, & \nu(1) &= 0. \end{aligned}$$

A trajectory that satisfies the system dynamics and the boundary conditions is said to be *feasible*, and the corresponding controls are said to be *admissible*. An optimal trajectory is one that minimizes an *objective function* and is feasible. Here we will use a common objective function: the integral of control effort squared. This cost function is desirable because it tends to produce smooth solution trajectories that are easily computed:

$$(2.3) \quad \min_{u(t), x(t), \nu(t)} \int_0^1 u^2(\tau) d\tau.$$

**2.2. Block-Move Example: Analytic Solution.** The solution to the simple block-moving trajectory optimization problem (2.1)–(2.3) is given below, with a full derivation shown in Appendix B:

$$(2.4) \quad u^*(t) = 6 - 12t, \quad x^*(t) = 3t^2 - 2t^3.$$

The analytic solution is found using principles from the calculus of variations. These methods convert the original optimization problem into a system of differential equations, which (in this special case) happen to have an analytic solution. It is worth noting that *indirect methods* for solving trajectory optimization work by using a similar principle: they analytically construct the necessary and sufficient conditions for optimality, and then solve them numerically. Indirect methods are briefly covered in section 9.4.

**2.3. Block-Move Example: Trapezoidal Collocation.** Now let's look at how to compute the optimal block-moving trajectory using trapezoidal collocation. We will need to convert the original continuous-time problem statement into a nonlinear program. First, we need to discretize the trajectory, which gives us a finite set of decision variables. This is done by representing the continuous position  $x(t)$  and velocity  $v(t)$  by their values at specific points in time, known as *collocation points*:

$$\begin{aligned} t &\rightarrow t_0 \dots t_k \dots t_N, \\ x &\rightarrow x_0 \dots x_k \dots x_N, \\ \nu &\rightarrow \nu_0 \dots \nu_k \dots \nu_N. \end{aligned}$$

Next, we need to convert the continuous system dynamics into a set of constraints that we can apply to the state and control at the collocation points. This is where trapezoid quadrature (also known as the trapezoid rule) is used. The key idea is that the change in state between two collocation points is equal to the integral of the system dynamics. That integral is then approximated using trapezoidal quadrature, as shown below, where  $h_k \equiv (t_{k+1} - t_k)$ :

$$\begin{aligned} \dot{x} &= \nu, \\ \int_{t_k}^{t_{k+1}} \dot{x} dt &= \int_{t_k}^{t_{k+1}} \nu dt, \\ x_{k+1} - x_k &\approx \frac{1}{2}(h_k)(\nu_{k+1} + \nu_k). \end{aligned}$$

Simplifying and then applying this to the velocity equation as well, we arrive at a set of equations that allow us to approximate the dynamics between each pair of collocation points. The constraints are known as *collocation constraints*, and these



equations are enforced on every segment,  $k = 0, \dots, (N - 1)$ , of the trajectory:

$$(2.5) \quad x_{k+1} - x_k = \frac{1}{2}(h_k)(\nu_{k+1} + \nu_k),$$

$$(2.6) \quad \nu_{k+1} - \nu_k = \frac{1}{2}(h_k)(u_{k+1} + u_k).$$

The boundary conditions are straightforward to handle: we simply apply them to the state at the initial and final collocation points:

$$(2.7) \quad \begin{array}{ll} x_0 = 0, & x_N = 1, \\ \nu_0 = 0, & \nu_N = 0. \end{array}$$

Finally, we approximate the objective function using trapezoid quadrature, converting it into a summation over each segment, using the control error at the collocation points:

$$(2.8) \quad \min_{u(t)} \int_{t_0}^{t_N} u^2(\tau) d\tau \approx \min_{u_0 \dots u_N} \sum_{k=0}^{N-1} \frac{1}{2}(h_k)(u_k^2 + u_{k+1}^2).$$

**2.4. Initialization.** Most nonlinear programming solvers require an initial guess. For easy problems, such as this one, a huge range of initial guesses will yield correct results from the nonlinear programming solver. However, for difficult problems a poor initial guess can cause the solver to get “stuck” on a bad solution or fail to converge entirely. Section 5.1 provides a detailed overview of methods for constructing an initial guess.

For the block-moving example, we will simply assume that the position of the block ( $x$ ) transitions linearly between the initial and final positions. We then differentiate this initial position trajectory to compute the velocity ( $\nu$ ) and force ( $u$ ) trajectories. Note that this choice of initial trajectory satisfies the system dynamics and position boundary condition, but it violates the velocity boundary condition:

$$(2.9) \quad x^{\text{init}}(t) = t,$$

$$(2.10) \quad \nu^{\text{init}}(t) = \frac{d}{dt}x^{\text{init}}(t) = 1,$$

$$(2.11) \quad u^{\text{init}}(t) = \frac{d}{dt}\nu^{\text{init}}(t) = 0.$$

Once we have an initial trajectory, we can evaluate it at each collocation point to obtain values to pass to the nonlinear programming solver:

$$(2.12) \quad x_k^{\text{init}} = t_k, \quad \nu_k^{\text{init}} = 1, \quad u_k^{\text{init}} = 0.$$

**2.5. Block-Move Example: Nonlinear Program.** We have used trapezoidal direct collocation to transcribe the continuous-time trajectory optimization problem into a nonlinear program (constrained parameter optimization problem) (2.5)–(2.8). Now, we just need to solve it! Section 9.12 provides a brief overview of software packages that solve this type of optimization problem.

In general, after performing direct transcription, a trajectory optimization problem is converted into a nonlinear programming problem. It turns out that, for this simple example, we actually get a quadratic program. This is because the constraints (2.5)–(2.7) are both linear, and the objective function (2.8) is quadratic. Solving a quadratic program is usually much easier than solving a nonlinear program.

**2.6. Block-Move Example: Interpolation.** Let's assume that you've solved the nonlinear program: you have a set of positions  $x_k$ , velocities,  $v_k$ , and controls  $u_k$  that satisfy the dynamics and boundary constraints and that minimize the objective function. All that remains is to construct a spline (piecewise polynomial function) that interpolates the solution trajectory between the collocation points. For trapezoidal collocation, it turns out that you use a linear spline for the control and a quadratic spline for the state. Section 3.4 provides more detailed discussion and derivation of these interpolation splines.

**3. Trapezoidal Collocation Method.** Now that we've seen how to apply trapezoidal collocation to a simple example, we'll take a deeper look at using trapezoidal collocation to solve a generic trajectory optimization problem.

Trapezoidal collocation works by converting a continuous-time trajectory optimization problem into a nonlinear program. This is done by using trapezoidal quadrature, also known as the trapezoid rule for integration, to convert each continuous aspect of the problem into a discrete approximation. In this section we will go through how this transformation is done for each aspect of a trajectory optimization problem.

**3.1. Trapezoidal Collocation: Integrals.** There are often integral expressions in trajectory optimization. Usually they are found in the objective function, but occasionally they are in the constraints as well. Our goal here is to approximate the continuous integral  $\int w(\cdot) dt$  as a summation  $\sum c_k w_k$ . The key concept here is that the summation only requires the value of the integrand  $w(t_k) = w_k$  at the collocation points  $t_k$  along the trajectory. This approximation is done by applying the trapezoid rule for integration between each collocation point, which yields following the equation, where  $h_k = t_{k+1} - t_k$  [6]:

$$(3.1) \quad \int_{t_0}^{t_F} w(\tau, \mathbf{x}(\tau), \mathbf{u}(\tau)) d\tau \approx \sum_{k=0}^{N-1} \frac{1}{2} h_k \cdot (w_k + w_{k+1}).$$

**3.2. Trapezoidal Collocation: System Dynamics.** One of the key features of a direct collocation method is that it represents the system dynamics as a set of constraints, known as *collocation constraints*. For trapezoidal collocation, the collocation constraints are constructed by writing the dynamics in integral form and then approximating that integral using trapezoidal quadrature [6]:

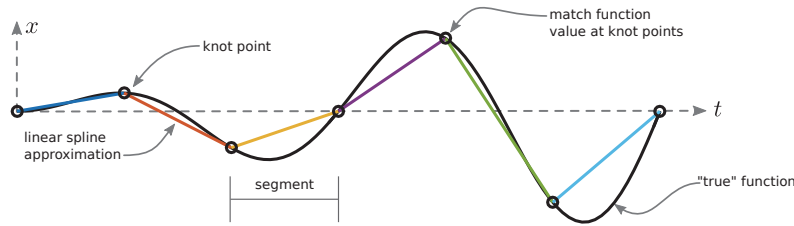
$$\begin{aligned} \dot{\mathbf{x}} &= \mathbf{f}, \\ \int_{t_k}^{t_{k+1}} \dot{\mathbf{x}} dt &= \int_{t_k}^{t_{k+1}} \mathbf{f} dt, \\ \mathbf{x}_{k+1} - \mathbf{x}_k &\approx \frac{1}{2} h_k \cdot (\mathbf{f}_{k+1} + \mathbf{f}_k). \end{aligned}$$

This approximation is then applied between every pair of collocation points:

$$(3.2) \quad \mathbf{x}_{k+1} - \mathbf{x}_k = \frac{1}{2} h_k \cdot (\mathbf{f}_{k+1} + \mathbf{f}_k), \quad k \in 0, \dots, (N-1).$$

Note that  $\mathbf{x}_k$  is a decision variable in the nonlinear program, while  $\mathbf{f}_k = \mathbf{f}(t_k, \mathbf{x}_k, \mathbf{u}_k)$  is the result of evaluating the system dynamics at each collocation point.

**3.3. Trapezoidal Collocation: Constraints.** In addition to the collocation constraints, which enforce the system dynamics, you might also have limits on the state and control, path constraints, and boundary constraints. These constraints are all



**Fig. 3** Function approximation using a linear spline.

handled by enforcing them at specific collocation points. For example, simple limits on state and control are approximated as follows:

$$(3.3) \quad \mathbf{x} < \mathbf{0} \quad \rightarrow \quad \mathbf{x}_k < \mathbf{0} \quad \forall k,$$

$$(3.4) \quad \mathbf{u} < \mathbf{0} \quad \rightarrow \quad \mathbf{u}_k < \mathbf{0} \quad \forall k.$$

Path constraints are handled similarly:

$$(3.5) \quad \mathbf{g}(t, \mathbf{x}, \mathbf{u}) < \mathbf{0} \quad \rightarrow \quad \mathbf{g}(t_k, \mathbf{x}_k, \mathbf{u}_k) < \mathbf{0} \quad \forall k.$$

Boundary constraints are enforced at the first and last collocation points:

$$(3.6) \quad \mathbf{h}(t_0, \mathbf{x}(t_0), \mathbf{u}(t_0)) < \mathbf{0} \quad \rightarrow \quad \mathbf{h}(t_0, \mathbf{x}_0, \mathbf{u}_0) < \mathbf{0}.$$

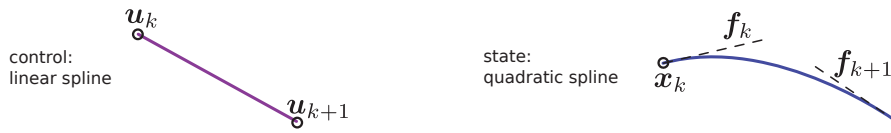
Finally, there are two notes of caution with regard to constraints. First, trajectory optimization problems with path constraints tend to be much harder to solve than those without. The details are beyond the scope of this paper, but are well covered by Betts [6]. Second, in trapezoidal collocation the boundaries of the trajectory are always collocation points. There are some methods, such as those presented in section 9.7, for which the trajectory boundaries are not collocation points. For these methods, special care must be taken when handling boundary constraints [3, 24].

**3.4. Trapezoidal Collocation: Interpolation.** Trapezoidal collocation works by approximating the control trajectory and the system dynamics as piecewise linear functions, also known as a *linear splines*, shown in Figure 3. When constructing a spline, the term *knot point* is used to denote any point that joins two polynomial segments. For trapezoidal collocation, the knot points of the spline are coincident with the collocation points.

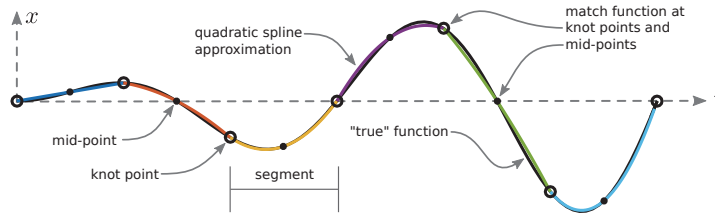
Let's start by constructing the control trajectory, which is a simple linear spline. We know both the time and the control at each knot point, so it is a simple matter to derive the expression for  $\mathbf{u}$  on the interval  $t \in [t_k, t_{k+1}]$ . To keep the mathematics readable, let's define  $\tau = t - t_k$  and  $h_k = t_{k+1} - t_k$ :

$$(3.7) \quad \mathbf{u}(t) \approx \mathbf{u}_k + \frac{\tau}{h_k} (\mathbf{u}_{k+1} - \mathbf{u}_k).$$

The state trajectory is represented by a *quadratic spline* — a piecewise quadratic function. This might seem confusing, but it follows directly from the collocation equations (3.2). The trapezoidal collocation equations are exact when the system



**Fig. 4** Illustration of the linear and quadratic spline segments that are used to approximate the control and state trajectories for trapezoidal collocation.



**Fig. 5** Function approximation using a quadratic spline. Notice that this approximation is far more accurate than the linear spline in Figure 3, for the same number of segments.

dynamics vary linearly between any two knot points, a fact that we use to approximate the dynamics over a single segment  $t \in [t_k, t_{k+1}]$ , as shown below:

$$(3.8) \quad \mathbf{f}(t) = \dot{\mathbf{x}}(t) \approx \mathbf{f}_k + \frac{\tau}{h_k} (\mathbf{f}_{k+1} - \mathbf{f}_k).$$

We are interested in  $\mathbf{x}$  and not  $\dot{\mathbf{x}}$ , so we integrate both sides of the equation to obtain a quadratic expression for the state:

$$(3.9) \quad \mathbf{x}(t) = \int \dot{\mathbf{x}}(t) d\tau \approx \mathbf{c} + \mathbf{f}_k \tau + \frac{\tau^2}{2h_k} (\mathbf{f}_{k+1} - \mathbf{f}_k).$$

We can solve for the constant of integration  $\mathbf{c}$  by using the value of the state at the boundary  $\tau = 0$  to obtain our final expression for the state:

$$(3.10) \quad \mathbf{x}(t) \approx \mathbf{x}_k + \mathbf{f}_k \tau + \frac{\tau^2}{2h_k} (\mathbf{f}_{k+1} - \mathbf{f}_k).$$

Figure 4 shows how a linear control segment and quadratic state segment are constructed. The spline equations (3.7) and (3.10) are used specifically for trapezoidal collocation, since there is a one-to-one correspondence between the collocation equations and the interpolating spline. In general, if the control is a spline of order  $n$ , then the state is represented by a spline of order  $n + 1$  [6].

**4. Hermite–Simpson Collocation Method.** The Hermite–Simpson collocation is similar to trapezoidal collocation, but it provides a solution that is higher-order accurate. This is because trapezoidal collocation approximates the objective function and system dynamics as piecewise linear functions, while Hermite–Simpson collocation approximates them as piecewise quadratic functions, as shown in Figure 5. An additional benefit of the Hermite–Simpson collocation method is that the state trajectory is a cubic Hermite spline, which has a continuous first derivative.

**4.1. Hermite–Simpson Collocation: Integrals.** Integral expressions are common in trajectory optimization problems, especially in the objective function. The Hermite–Simpson collocation method approximates these integrals using Simpson quadrature. Simpson quadrature, also known as Simpson’s rule for integration, works by approximating the integrand of the integral as a piecewise quadratic function. This approximation is given below and derived in Appendix C:

$$\int_{t_0}^{t_F} w(\tau) d\tau \approx \sum_{k=0}^{N-1} \frac{h_k}{6} (w_k + 4w_{k+\frac{1}{2}} + w_{k+1}).$$

**4.2. Hermite–Simpson Collocation: System Dynamics.** In any collocation method the *collocation constraints* are the set of constraints that are constructed to approximate the system dynamics. In the Hermite–Simpson collocation method we construct these constraints by rewriting the system dynamics in integral form: the change in state between any two knot points  $t_k$  should be equal to the integral of the system dynamics  $\mathbf{f}(\cdot)$  between those points:

$$(4.1) \quad \dot{\mathbf{x}} = \mathbf{f},$$

$$(4.2) \quad \int_{t_k}^{t_{k+1}} \dot{\mathbf{x}} dt = \int_{t_k}^{t_{k+1}} \mathbf{f} dt.$$

The transcription from continuous dynamics to a set of collocation equations occurs when we approximate the continuous integral in (4.2) with Simpson quadrature and apply it between every pair of knot points:

$$(4.3) \quad \mathbf{x}_{k+1} - \mathbf{x}_k = \frac{1}{6} h_k (\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} + \mathbf{f}_{k+1}).$$

For Hermite–Simpson collocation we actually need a second collocation equation, in addition to (4.3), to enforce the dynamics. This is because the dynamics at the midpoint of the segment  $\mathbf{f}_{k+\frac{1}{2}}$  are a function of the state  $\mathbf{x}_{k+\frac{1}{2}}$ , which is not known a priori. We can compute the state at the midpoint by constructing an interpolant for the state trajectory (see section 4.4) and then evaluating it at the midpoint of the interval:

$$(4.4) \quad \mathbf{x}_{k+\frac{1}{2}} = \frac{1}{2} (\mathbf{x}_k + \mathbf{x}_{k+1}) + \frac{h_k}{8} (\mathbf{f}_k - \mathbf{f}_{k+1}).$$

This second collocation equation (4.4) is special in that it can be computed explicitly in terms of the state at the knot points. Thus, it is possible to combine both (4.4) and (4.3) into a single complicated collocation constraint. When transcription of the system dynamics is performed using this single collocation constraint, the resulting formulation is said to be in *compressed form*. An alternative implementation is to create an additional decision variable for the state at the midpoint  $\mathbf{x}_{k+\frac{1}{2}}$ , and then use both (4.3) and (4.4) as constraint equations. When the collocation equations are formulated using this pair of constraints they are said to be in *separated form*.

There are a variety of trade-offs between the separated and compressed forms of Hermite–Simpson collocation, which are covered in detail in [6]. The general rule is that the separated form is better when working with a smaller number of segments, while the compressed form is better when the number of segments is large. Both constraint equations (4.3) and (4.4) can be found in Betts’s book [6].

**4.3. Hermite–Simpson Collocation: Constraints.** In addition to the collocation constraints, which enforce the system dynamics, you might also have limits on the state and control, path constraints, and boundary constraints. These constraints are all handled by enforcing them at specific collocation points. For example, simple limits on state and control are approximated as follows:

$$(4.5) \quad \mathbf{x} < \mathbf{0} \quad \rightarrow \quad \begin{aligned} \mathbf{x}_k &< \mathbf{0}, \\ \mathbf{x}_{k+\frac{1}{2}} &< \mathbf{0}, \end{aligned}$$

$$(4.6) \quad \mathbf{u} < \mathbf{0} \quad \rightarrow \quad \begin{aligned} \mathbf{u}_k &< \mathbf{0}, \\ \mathbf{u}_{k+\frac{1}{2}} &< \mathbf{0}. \end{aligned}$$

Path constraints are handled similarly: they are applied at all collocation points, as shown below:

$$(4.7) \quad \mathbf{g}(t, \mathbf{x}, \mathbf{u}) < \mathbf{0} \quad \rightarrow \quad \begin{aligned} \mathbf{g}(t_k, \mathbf{x}_k, \mathbf{u}_k) &< \mathbf{0}, \\ \mathbf{g}(t_{k+\frac{1}{2}}, \mathbf{x}_{k+\frac{1}{2}}, \mathbf{u}_{k+\frac{1}{2}}) &< \mathbf{0}. \end{aligned}$$

Boundary constraints are enforced at the first and last knot points:

$$(4.8) \quad \mathbf{h}(t_0, \mathbf{x}(t_0), \mathbf{u}(t_0)) < \mathbf{0} \quad \rightarrow \quad \mathbf{h}(t_0, \mathbf{x}_0, \mathbf{u}_0) < \mathbf{0}.$$

Just like in trapezoidal collocation, trajectory optimization problems with path constraints tend to be much harder to solve than those without [6]. Additionally, in Hermite–Simpson collocation the boundaries of the trajectory are always collocation points. There are some methods, such as those presented in section 9.7, for which the trajectory boundaries are not collocation points. For these methods, special care must be taken when handling boundary constraints. [3, 24].

**4.4. Hermite–Simpson Collocation: Interpolation.** After we’ve solved the nonlinear program, we know the value of the state and control trajectories at each collocation point. The next step is to construct a continuous trajectory to interpolate the solution between the collocation points. Just like with trapezoidal collocation, we will use a polynomial interpolant that is derived from the collocation equations.

Hermite–Simpson collocation works by using Simpson quadrature to approximate each segment of the trajectory. As shown in Appendix C, Simpson quadrature uses a quadratic segment, fitted through three uniformly spaced points, to approximate the integrand. In this case, we are approximating both the control and the system dynamics as quadratic over each segment of the trajectory.

The general equation for quadratic interpolation is given in *Numerical Recipes in C* [50] and reproduced below for a curve  $\mathbf{u}(t)$  that passes through three points,  $(t_A, \mathbf{u}_A)$ ,  $(t_B, \mathbf{u}_B)$ , and  $(t_C, \mathbf{u}_C)$ :

$$(4.9) \quad \mathbf{u}(t) = \frac{(t - t_B)(t - t_C)}{(t_A - t_B)(t_A - t_C)} \mathbf{u}_A + \frac{(t - t_A)(t - t_C)}{(t_B - t_A)(t_B - t_C)} \mathbf{u}_B + \frac{(t - t_A)(t - t_B)}{(t_C - t_A)(t_C - t_B)} \mathbf{u}_C.$$

For our specific case, we can simplify this equation quite a bit, since our points are uniformly spaced. Let’s start by using points  $k$ ,  $k + \frac{1}{2}$ , and  $k + 1$  in place of  $A$ ,  $B$ , and  $C$ . Next, recall from previous sections that  $h_k = t_{k+1} - t_k$ ,  $t_{k+\frac{1}{2}} = \frac{1}{2}(t_k + t_{k+1})$ , and  $\tau = t - t_k$ . After making these substitutions and doing some algebra, we can arrive at the following simplified equation for interpolating the control trajectory:

$$(4.10) \quad \mathbf{u}(t) = \frac{2}{h_k^2}(\tau - \frac{h_k}{2})(\tau - h_k) \mathbf{u}_k - \frac{4}{h_k^2}(\tau)(\tau - h_k) \mathbf{u}_{k+\frac{1}{2}} + \frac{2}{h_k^2}(\tau)(\tau - \frac{h_k}{2}) \mathbf{u}_{k+1}.$$



**Fig. 6** Illustration of the quadratic and cubic spline segments that are used to approximate the control and state trajectories for Hermite-Simpson collocation.

Hermite-Simpson collocation also represents the system dynamics  $\mathbf{f}(\cdot) = \dot{\mathbf{x}}$  using quadratic polynomials over each segment. As a result, the quadratic interpolation formula that we have developed for the control trajectory can also be applied to the system dynamics:

$$(4.11) \quad \mathbf{f}(t) = \dot{\mathbf{x}} = \frac{2}{h_k^2}(\tau - \frac{h_k}{2})(\tau - h_k)\mathbf{f}_k - \frac{4}{h_k^2}(\tau)(\tau - h_k)\mathbf{f}_{k+\frac{1}{2}} + \frac{2}{h_k^2}(\tau)(\tau - \frac{h_k}{2})\mathbf{f}_{k+1}.$$

Usually we are interested in obtaining an expression for the state trajectory  $\mathbf{x}(t)$  rather than its derivative  $\dot{\mathbf{x}}(t)$ . To find the state trajectory, we simply integrate (4.11), after rearranging it into standard polynomial form:

$$(4.12) \quad \begin{aligned} \mathbf{x}(t) &= \int \dot{\mathbf{x}} dt \\ &= \int \left[ \mathbf{f}_k + \left( -3\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} - \mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right) + \left( 2\mathbf{f}_k - 4\mathbf{f}_{k+\frac{1}{2}} + 2\mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right)^2 \right] dt. \end{aligned}$$

We can compute the integral using basic calculus and then solve for the constant of integration using the boundary condition  $\mathbf{x}(t_k) = \mathbf{x}_k$ . The resulting expression is given below, which allows us to interpolate the state trajectory:

$$(4.13) \quad \begin{aligned} \mathbf{x}(t) &= \mathbf{x}_k + \mathbf{f}_k \left( \frac{\tau}{h_k} \right) + \frac{1}{2} \left( -3\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} - \mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right)^2 \\ &\quad + \frac{1}{3} \left( 2\mathbf{f}_k - 4\mathbf{f}_{k+\frac{1}{2}} + 2\mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right)^3. \end{aligned}$$

The interpolants for the state and control trajectories are illustrated in Figure 6.

**5. Practical Considerations.** This section of the paper provides an overview of several important topics that are related to trajectory optimization in general, rather than to some specific method. We start with some practical suggestions about how to initialize trajectory optimization problems, followed by two sections that explain how to check the accuracy of a given solution. We conclude by looking at some common bugs that show up in trajectory optimization code and how to go about fixing them.

**5.1. Initialization.** Nearly all trajectory optimization techniques require a good initial guess to begin the optimization. In the best case, a good initialization will ensure that the solver rapidly arrives at the globally optimal solution. In the worst case, a bad initialization can cause the nonlinear programming solver to fail to solve an otherwise correct optimization problem.

To understand these concepts, let's use an analogy: imagine that the optimization is trying to get to the top of a hill. If the landscape is simple, with only one hill, then it doesn't matter where the optimization starts: it can go uphill until it finds the solution. What happens if there are two different hills and one is higher? Then there will be some starting points where going uphill will only get you to the lower of the two hills. In this case, the optimization will know that it got to the top of the hill, but it won't know that there is an even higher hill somewhere else.

Just like in the simple hill-climbing analogy, the choice of initial guess can affect which local minimum the optimization eventually converges to. The presence of constraints makes it even worse: there might be some starting points from which the optimization cannot even find a feasible solution. This is a fundamental problem with nonlinear programming solvers: they cannot always find a solution, and if they do find a solution, it is only guaranteed to be locally optimal.

The best initializations for trajectory optimization usually require some problem-specific knowledge, but there are a few general approaches that can be useful. In this way, initialization is more of an art than a science. One good practice is to try several different initialization strategies and check that they all converge to the same solution. See section 5.4 for some debugging suggestions to help determine whether a solution is converging correctly.

One of the simplest initialization techniques is to assume that the trajectory is a straight line in state space between the initial and final states. This approach is easy to implement, and will often work well, especially for simple boundary value problems.

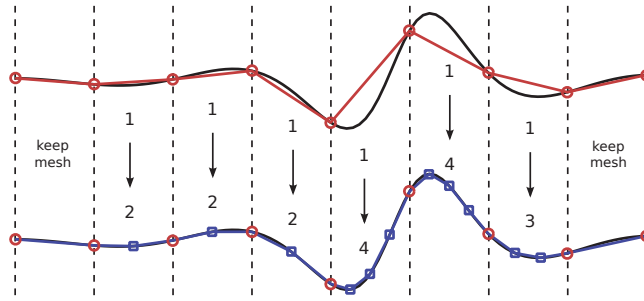
If you have a rough idea of what the behavior should look like, then you can put that in as the initial guess. For example, if you want a robot to do a back-flip, sketch out the robot at a few points throughout the back-flip, figure out the points in state space for each configuration, and then use linear interpolation between those points.

For complicated problems, a more principled approach might be required. Our favorite technique is to simplify the trajectory optimization problem until we can find a reasonable solution using a simple initialization technique. Then we use the solution of the simplified problem to initialize the original problem. If this doesn't work, then we simply construct a series of trajectory optimization problems, each of which is slightly closer to the desired problem and uses the previous solution as the initial guess.

For example, let's say that you want to find a minimum-work trajectory for a walking robot. This objective function is challenging to optimize (see section 8), and there are some difficult nonlinear constraints: foot clearance, contact forces, and walking speed. Start by replacing the objective function with something simple: a minimum torque-squared objective (like the five-link biped example; see section 7). Next, remove most of the constraints and replace the nonlinear dynamics with simple kinematics (joint acceleration = joint torque). Solve this problem, and then use the solution to initialize a slightly harder version of the problem where you've added back in some of the constraints. You can then repeat this process until you have a solution to your original trajectory optimization problem. This process is also a good way to find bugs in both your problem statement and the code.

**5.2. Mesh Refinement.** The direct transcription process approximates a trajectory using polynomial splines, which allows the trajectory optimization problem to be converted into a nonlinear program. The collocation constraints in the result-





**Fig. 7** Illustration of mesh refinement by subdividing segments. The number of subsegments is determined by the peak error in each segment.

ing nonlinear program act as implicit Runge–Kutta integration schemes [6]. Just like any integration scheme, there are numerical errors associated with the choices of time step and method order. Using short time steps (dense mesh) and a high-order method will result in an accurate approximation, but at a significant computational cost.

*Mesh refinement* is the process by which a trajectory optimization problem is solved on a sequence of different collocation meshes, also known as *collocation grids*. The mesh (grid) refers to the choice of discretization along the trajectory. Generally, the first mesh is coarse, with a small number of collocation points and (or) a lower-order collocation method. Subsequent meshes have more points and (or) higher-order collocation methods. This iterative strategy is implemented to obtain the most accurate solution with the least amount of computational effort: the solutions using the initial meshes are easy to solve but inaccurate, while the solutions on subsequent meshes are more costly to compute but more accurate.

Figure 7 shows a simple example of how the mesh for a linear spline might be refined to produce a more accurate representation by adding a small number of points. The segments with a small error are left unchanged, while segments with more error are subdivided into 2, 3, or 4 subsegments for the next iteration.

In more sophisticated mesh-refinement methods, the accuracy of a given segment might be improved by subdividing it or by increasing the polynomial order inside the segment. Such algorithms are referred to as *hp-adaptive meshing*. The decision to subdivide the mesh or to increase the polynomial order is made by examining the error profile within a single segment. If there is a spike in the error, then the segment is subdivided, otherwise the polynomial order is increased, for example, switching from trapezoidal to Hermite–Simpson collocation [16], [45], and [6].

**5.3. Error Analysis.** There are two types of numerical errors that are present in the solution of a trajectory optimization problem: transcription errors and errors in the solution to the nonlinear program. Here we will focus on the accuracy of the transcription process, quantifying the error that was introduced by the choice of discretization (both method and grid). We can then use these error estimates to compute a new discretization, as described in section 5.2.

There are many possible error metrics for trajectory optimization [6]. Here we will construct an error estimate based on how well the candidate trajectory satisfies the system dynamics between the collocation points. The logic here is that if the system

dynamics are accurately satisfied between the collocation points, then the polynomial spline is an accurate representation of the system, which would then imply that the nonlinear program is an accurate representation of the original trajectory optimization problem.

We do not know the true solution  $\mathbf{x}^*(t)$ ,  $\mathbf{u}^*(t)$  of the trajectory optimization problem, but we do know that it must precisely satisfy the system dynamics

$$\dot{\mathbf{x}}^*(t) = \mathbf{f}(t, \mathbf{x}^*(t), \mathbf{u}^*(t)).$$

From this, we can construct an expression for the error in the solution to the system dynamics along the candidate trajectory. It is important that the solution  $\mathbf{x}(t)$ ,  $\mathbf{u}(t)$  is evaluated using method consistent interpolation [6]:

$$\boldsymbol{\varepsilon}(t) = \dot{\mathbf{x}}(t) - \mathbf{f}(t, \mathbf{x}(t), \mathbf{u}(t)).$$

This error  $\boldsymbol{\varepsilon}(t)$  will be zero at each collocation point and nonzero elsewhere. We can compute the integral of the error  $\boldsymbol{\varepsilon}(t)$  numerically to determine how far the candidate solution (polynomial spline) may have deviated from the true solution along each dimension of the state. The following expression for the error is typically evaluated using Rhomberg quadrature [6]:

$$\boldsymbol{\eta}_k = \int_{t_k}^{t_{k+1}} |\boldsymbol{\varepsilon}(\tau)| d\tau.$$

Once you compute the error in each state over each segment of the trajectory, you can use it to determine how to remesh the trajectory (section 5.2) so that the optimization converges to an optimal solution that satisfies the continuous dynamics. See [6] and [16] for additional details about how to compute error estimates and perform mesh refinement.

**5.4. Debugging Your Code.** There are many ways that trajectory optimization can go wrong. In this section, we discuss some common bugs that find their way into code and a few techniques for locating and fixing them. Betts [6] also provides a good list of debugging suggestions.

One particularly tricky type of bug occurs when there is a family of optimal solutions, rather than a single unique solution. This causes a failure to converge because the optimization is searching for a locally optimal solution, which it never finds because many solutions are equally good. The fix is to modify the problem statement so that there is a unique solution. One simple way to do this is to add a small regularization term to the cost function, such as the integral of control squared along the trajectory. This puts a shallow bowl in the objective function, forcing a unique solution. Trajectory optimization problems with nonunique solutions often have *singular arcs*, which occur when the optimal control is not uniquely defined by the objective function. A more formal treatment of singular arcs is provided in [5] and [6].

A trajectory optimization problem with a nonsmooth *solution* (control) might cause the nonlinear program to converge very slowly. This occurs in our final example: finding the minimal work trajectory to move a block between two points (section 8). There are three basic ways to deal with a discontinuous solution (control). The first is to do mesh refinement (section 5.2) so that there are many short segments near the discontinuity. The second is to slightly modify the problem, typically by introducing

a smoothing term, such that the solution is numerically stiff but not discontinuous. This second approach was used in [55]. The third approach is to solve the problem using a multiphase method (see section 9.9), such that the control in each phase of the trajectory is continuous, and discontinuities occur between phases.

Another common cause of poor convergence in the nonlinear programming solver occurs when the objective and (or) constraint functions have discontinuous gradients (see section 5.5). There are many sources of inconsistency that find their way into trajectory optimization problems: discontinuous functions (`abs()`, `min()`, `max()`...), random number generators, variable step (adaptive) integration, iterative root finding, and table interpolation. All of these will cause significant convergence problems if placed inside a standard nonlinear programming solver. Section 5.5 covers some methods for handling inconsistent functions.

If the nonlinear programming solver returns saying that the problem is infeasible, there are two possible scenarios. The first is that your problem statement is actually impossible: you have contradictory constraints. In this case, you can often find some clues by looking at final point in the nonlinear programming solution (the best of the infeasible trajectories). What constraints are active? Is the trajectory right on top of your initial guess? Is it running into an actuator limit? You can also debug this type of failure by removing constraints from the problem until it converges and then adding them back one at a time.

The second cause of an infeasible report from a nonlinear programming solver is when a complicated optimization problem is initialized with a poor guess. In this case, the optimization gets stuck in a “bad” local minimum, that has no feasible solution. The best fix in this case is to use the methods discussed in section 5.1 to compute a better initialization.

It is challenging to determine if a candidate solution is at a global or a local minimum. In both cases the nonlinear programming solver will report success. In general, there is no rigorous way to determine if you have the globally optimal solution, but there are many effective heuristics. One such heuristic is to run the optimization from a wide variety of initial guesses. If most of the guesses converge to the same solution, and it is better than all others found, there is a good chance that this is the globally optimal solution. Another such heuristic is to use different transcription methods and check that all methods all converge to the same solution.

**5.5. Consistent Functions.** Direct transcription solves a trajectory optimization problem by converting it to a nonlinear program. Most nonlinear programming solvers, such as SNOPT [25], IPOPT [10], and FMINCON [37], require that the user-defined objective and constraint functions be *consistent*. A function is consistent if it performs the exact same sequence of arithmetic operations on each call [6]. This is essentially like saying that the function must have no logical branches, be deterministic, and have outputs that vary smoothly with the inputs.

For example, the `abs()` function is not consistent, because of the discontinuity in the derivative at the origin. The functions `min()` and `max()` are also not consistent. Imagine a function with two widely spaced peaks. A small change in the shape of the function could cause the maximum value to jump from one peak  $f(x_1)$  to a second peak  $f(x_2)$ . The problem here is in the gradients: when the peak moves, the gradient  $\frac{\partial f}{\partial x_1}$  jumps to zero, and the gradient  $\frac{\partial f}{\partial x_2}$  jumps from zero to some nontrivial value.

There is a neat trick that allows many inconsistent functions (such as `abs()`, `min()`, and `max()`) to be implemented consistently by introducing extra decision

variables (known as *slack variables*) and constraints into the problem. An example is given in section 8, showing how to correctly implement the `abs()` function. This topic is also covered by Betts [6]. An alternative way to handle such functions is to use smoothing, which is also demonstrated in the block-moving example in section 8.

Another place where inconsistency shows up is when a function has an internal iteration loop, such as in root finding or in a variable-step integration method. The correct way to implement a root-finding method inside an optimization is to use a fixed number of iterations. Likewise, a variable-step integration method should be replaced with a fixed-step method [6].

There are many situations where evaluating the dynamics or constraint functions require a table look-up, for example, computing the lift force generated by an airfoil. Linear interpolation of a table has a discontinuous derivative when switching between two different table entries. The fix is to switch to an interpolation scheme that has continuous derivatives. Continuous first derivatives are required by most solvers when computing gradients (first partial derivatives). Solvers that compute both gradients and Hessians (second partial derivatives) will require continuous second derivatives [6].

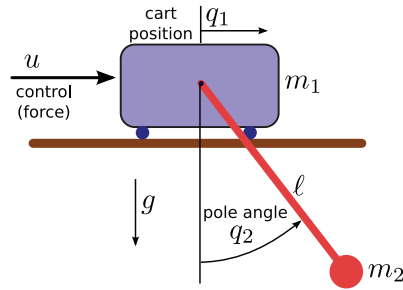
One final source of inconsistency is the use of a time-stepping simulator such as Bullet [14] or Box2d [12] to compute the system dynamics. The contact solvers in these simulators are inconsistent, which then leads to poor convergence in the nonlinear program. The best way to address this source of inconsistency is to rewrite the system dynamics. If the sequence of contacts is known and the dynamics can be described as a simple hybrid system, then you can use multiphase trajectory optimization to compute the solution (see section 9.9). For more complex systems where the contact sequence is unknown, you can use through-contact trajectory optimization to compute the solution [40, 48] (see section 9.10). If you need to use the time-stepping simulator, then you can use some of the methods developed by the computer graphics community [60, 61, 1, 34].

**6. Cart-Pole Swing-Up Example.** The cart-pole system is commonly used as a teaching tool in both introductory controls and trajectory optimization. The system comprises a cart that travels along a horizontal track and a pendulum that hangs freely from the cart. There is a motor that drives the cart forward and backward along the track. It is possible to move the cart in such a way that the pendulum, initially hanging below the cart at rest, is swung up to a point of inverted balance above the cart. In this section, we will use direct collocation to compute the minimum-force trajectory to perform this so-called “swing-up” maneuver.

**6.1. Cart-Pole Example: System Dynamics.** The cart-pole is a second-order dynamical system and its equations of motion can be derived using methods found in any undergraduate dynamics text book. The dynamics of this system are simple enough to derive by hand, although for more complicated systems it is generally a good idea to use a computer algebra package instead.

The position of the cart is given by  $q_1$ , the angle of the pole is given by  $q_2$ , and the control force is given by  $u$ . The masses of the cart and pole are given by  $m_1$  and  $m_2$ , respectively, and the length of the pole and acceleration due to gravity are  $\ell$  and  $g$ , as shown in Figure 8. The dynamics ( $\ddot{q}_1$  and  $\ddot{q}_2$ ) for the cart-pole system are

$$(6.1) \quad \ddot{q}_1 = \frac{\ell m_2 \sin(q_2) \dot{q}_2^2 + u + m_2 g \cos(q_2) \sin(q_2)}{m_1 + m_2 (1 - \cos^2(q_2))},$$



**Fig. 8** Physical model for the cart-pole example problem. The pendulum is free to rotate about its support point on the cart.

$$(6.2) \quad \ddot{q}_2 = - \frac{\ell m_2 \cos(q_2) \sin(q_2) \dot{q}_2^2 + u \cos(q_2) + (m_1 + m_2) g \sin(q_2)}{\ell m_1 + \ell m_2 (1 - \cos^2(q_2))}.$$

All standard trajectory optimization methods require that the dynamics of the system be in first-order form. This is accomplished by including both the minimal coordinates ( $q_1$  and  $q_2$ ) and their derivatives in the state. Note that  $\ddot{q}_1$  and  $\ddot{q}_2$  are defined in (6.1) and (6.2):

$$\mathbf{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}, \quad \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, u) = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix}.$$

**6.2. Cart-Pole Example: Objective Function.** For this example we will use one of the more common objective functions in trajectory optimization: the integral of the actuator-effort (control) squared:

$$(6.3) \quad J = \int_0^T u^2(\tau) d\tau.$$

This objective function (6.3) tends to produce smooth trajectories, which are desirable for two key reasons. The first is that most transcription methods assume that the solution to the trajectory optimization problem is well approximated by a polynomial spline. Thus, a problem with a solution that is smooth will be solved more quickly and accurately than a problem with a nonsmooth solution. The second benefit of smooth trajectories is that they tend to be easier to stabilize with conventional controllers when implemented on a real system.

**6.3. Cart-Pole Example: Boundary Constraints.** Many trajectory optimization problems include *boundary constraints*, which restrict the state of the system at the boundaries of the trajectory. Here we will restrict the full state of the cart-pole system at both the initial and final points on the trajectory. Let's suppose that we want the cart to start in the center of the rails and translate a distance  $d$  during its swing-up maneuver. The (constant) boundary constraints for this situation are given

by

$$\begin{aligned} q_1(t_0) &= 0, & q_1(t_F) &= d, \\ q_2(t_0) &= 0, & q_2(t_F) &= \pi, \\ \dot{q}_1(t_0) &= 0, & \dot{q}_1(t_F) &= 0, \\ \dot{q}_2(t_0) &= 0, & \dot{q}_2(t_F) &= 0. \end{aligned}$$

**6.4. Cart-Pole Example: State and Control Bounds.** The cart-pole swing-up problem has a few simple constraints. First, let's look at the state. The cart rides on a track which has a finite length, so we need to include a simple constraint that limits the horizontal range of the cart. Additionally, we will restrict the motor force to some maximal force in each direction:

$$\begin{aligned} -d_{\max} &\leq q_1(t) \leq d_{\max}, \\ -u_{\max} &\leq u(t) \leq u_{\max}. \end{aligned}$$

**6.5. Cart-Pole Example: Trapezoidal Collocation.** We can collect all of the equations in this section and combine them with the trapezoidal collocation method from section 3, and write down the cart-pole swing-up problem as a nonlinear program:

$$\begin{aligned} &\text{minimize} \\ (6.4) \quad &J = \sum_{k=0}^{N-1} \frac{h_k}{2} (u_k^2 + u_{k+1}^2), && \text{objective function,} \\ &\text{with decision variables} \\ (6.5) \quad &\mathbf{x}_0, \dots, \mathbf{x}_N \quad u_0, \dots, u_N, \\ &\text{subject to} \\ (6.6) \quad &\frac{1}{2} h_k (\mathbf{f}_{k+1} + \mathbf{f}_k) = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad k \in 0, \dots, (N-1), && \text{collocation constraints,} \\ (6.7) \quad &-d_{\max} \leq q_1 \leq d_{\max}, && \text{path constraints,} \\ (6.8) \quad &-u_{\max} \leq u \leq u_{\max}, && \text{path constraints,} \\ (6.9) \quad &\mathbf{x}_0 = \mathbf{0}, \quad \mathbf{x}_N = [d, \pi, 0, 0]^T, && \text{boundary constraints.} \end{aligned}$$

Note that  $h_k = t_{k+1} - t_k$ . Here, we will use a uniform grid, so  $t_k = k \frac{T}{N}$ , where  $N$  is the number of segments used in the transcription. In general, you could solve this problem on an arbitrary grid; in other words, each  $h_k$  could be different.

**6.6. Cart-Pole Example: Hermite–Simpson Collocation.** We can also use Hermite–Simpson collocation (section 4) to construct a nonlinear program for the cart-pole swing-up problem. This is similar to the trapezoidal collocation, but it uses a quadratic (rather than linear) spline to approximate the dynamics and control. Here we will use the separated form of the Hermite–Simpson method, which requires

including collocation points for the state and control at the midpoint of each segment  $t_{k+\frac{1}{2}}$  (see section 4.2):

minimize

(6.10)

$$J = \sum_{k=0}^{N-1} \frac{h_k}{6} (u_k^2 + 4u_{k+\frac{1}{2}}^2 + u_{k+1}^2), \quad \text{objective function,}$$

with decision variables

$$\mathbf{x}_0, \mathbf{x}_{0+\frac{1}{2}}, \dots, \mathbf{x}_N, \quad u_0, u_{0+\frac{1}{2}}, \dots, u_N,$$

subject to

(6.11)

$$\mathbf{x}_{k+\frac{1}{2}} = \frac{1}{2}(\mathbf{x}_k + \mathbf{x}_{k+1}) + \frac{h_k}{8}(\mathbf{f}_k - \mathbf{f}_{k+1}), \quad k \in 0, \dots, (N-1), \quad \text{interpolation constraints,}$$

(6.12)

$$\frac{h_k}{6}(\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} + \mathbf{f}_{k+1}) = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad k \in 0, \dots, (N-1), \quad \text{collocation constraints,}$$

(6.13)

$$-d_{\max} \leq q_1 \leq d_{\max}, \quad \text{path constraints,}$$

(6.14)

$$-u_{\max} \leq u \leq u_{\max}, \quad \text{path constraints,}$$

(6.15)

$$\mathbf{x}_0 = \mathbf{0}, \quad \mathbf{x}_N = [d, \pi, 0, 0]^T, \quad \text{boundary constraints.}$$

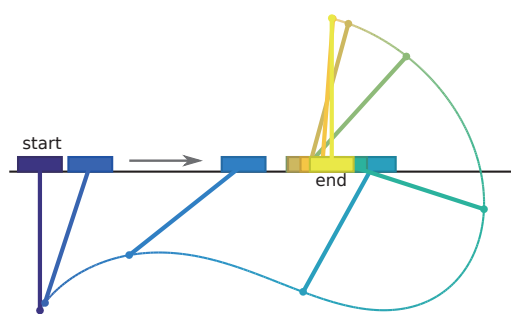
**6.7. Cart-Pole Example: Initialization.** The cart-pole swing-up problem is a boundary value problem: we are given the initial and final states, and our task is to compute an optimal trajectory between those two points. An obvious (and simple) initial guess is that the system moves linearly between the initial and final states with zero control effort. This simple guess works well for this problem, despite its failure satisfy the system dynamics:

$$(6.16) \quad \mathbf{x}_{\text{guess}}(t) = \frac{t}{T} \begin{bmatrix} d \\ \pi \\ 0 \\ 0 \end{bmatrix}, \quad u_{\text{guess}}(t) = 0.$$

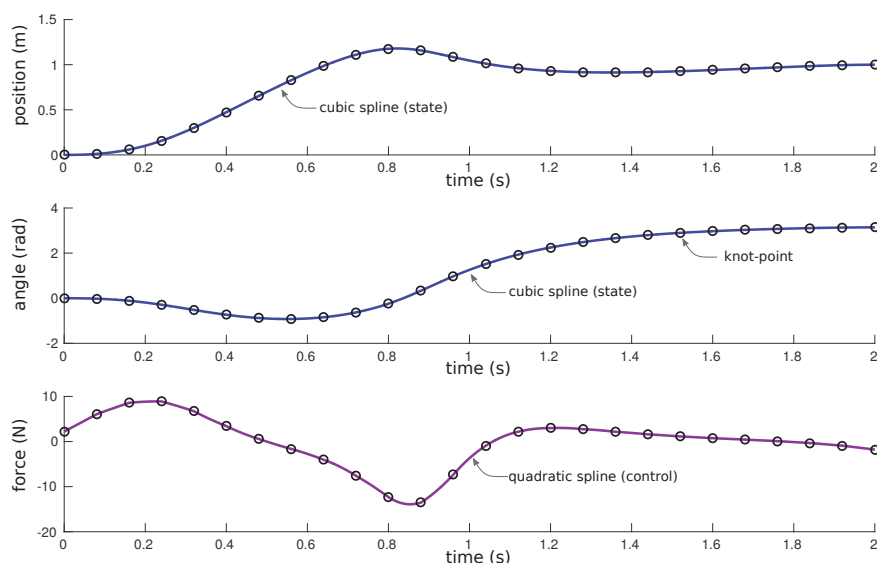
Additionally, we will start with a uniform grid, such that  $t_k = k\frac{T}{N}$ . The initial guess for each decision variable in the nonlinear program is then computed by evaluating (6.16) at each knot point  $t_k$  (and the midpoint  $t_{k+\frac{1}{2}}$  for Hermite–Simpson collocation).

**6.8. Cart-Pole Example: Results.** Here we show the optimal swing-up trajectory for the cart-pole system, computed using Hermite–Simpson collocation with 25 trajectory segments. The set of parameters that we use is given in Appendix E.1. We computed the solution in MATLAB, on a regular desktop computer,<sup>1</sup> using the

<sup>1</sup>Processor: 3.4GHz quad-core Intel i5-3570K.



**Fig. 9** Illustration of the optimal trajectory for the cart-pole swing-up example. The frames are uniformly spaced in time, moving from blue (dark) to yellow (light) as the trajectory progresses.



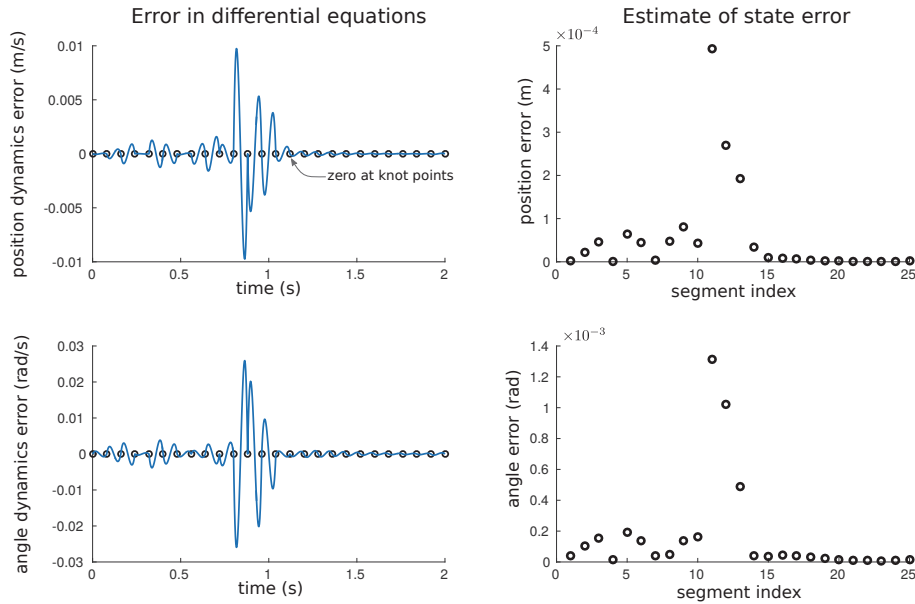
**Fig. 10** Plots showing the optimal trajectory for the cart-pole swing-up example.

code provided in the electronic supplement (see Appendix A). The nonlinear program was solved by FMINCON in 5.91 seconds (71 iterations) using default convergence settings.

Figure 9 shows a stop-action animation of the swing-up maneuver, with uniformly spaced frames. The same solution is shown in Figure 10 as plots of state and control versus time. Finally, Figure 11 shows the error estimates along the trajectory.

Notice that the error metrics in both the differential equations and the state increase noticeably near the middle of the trajectory. At this point, the system is changing rapidly as the pole swings up, and the uniform grid has difficulty approximating the system dynamics. A more sophisticated method would compute a new grid, in which the trajectory segments are shorter near this point where the system is rapidly changing.





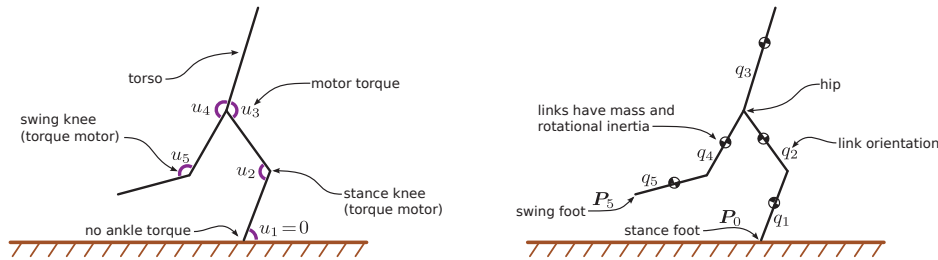
**Fig. 11** Plots showing the error in the system dynamics along the optimal trajectory for the cart-pole swing-up example. The plots on the left show the error in the differential equations, while the plots on the right show the integral of that error over each segment.

We selected parameters for this problem to ensure it is well behaved: we can make small changes to the initial guess or the direct transcription method and obtain the same basic answer. If we can change some of the problem parameters it can make things more difficult. For example, if we increase the duration  $T$ , this causes the optimal solution to include several swings back-and-forth before the final swing-up. As a result, the optimization problem has many local minima, one for each (incorrect) number of swings back and forth. Another way to make the optimization more challenging is to reduce the actuator limits  $u_{\max}$ . If these limits are made small enough, then the optimal solution will no longer be smooth. To solve it, we would need to remesh the discretization (time) grid to place additional points near the discontinuities in the force trajectory. An alternative way to address the discontinuity in the control would be to rewrite the problem as a multiphase problem, but this is beyond the scope of this paper.

**7. Five-Link Biped Example.** In this section we will use trajectory optimization to find a periodic walking gait for a five-link (planar) biped walking model. This model is commonly used when studying bipedal walking robots [67, 49, 43, 27, 54, 66]. For this example, we will use the model developed in [66], with parameters that are selected to match the walking robot RABBIT [13] and given in Appendix E.2.

We will assume that the robot is left-right symmetric, so we can search for a periodic walking gait using a single step (as opposed to a stride, which would consist of two steps). A periodic walking gait means that joint trajectories (torques, angles, and rates) are the same on each successive step. We will be optimizing the walking gait such that it minimizes the integral of torque-squared along the trajectory.

**7.1. Five-Link Biped: Model.** Figure 12 shows the five-link biped model as it takes a step. This model consists of a torso connected to two legs, each of which has



**Fig. 12** Illustration of the five-link biped model. We assume that the biped is a planar kinematic chain, with each joint connected to its parent by an ideal revolute joint and torque source. The biped is underactuated, because the stance ankle has no motor.

an upper and a lower link. The *stance* leg is supporting the weight of the robot, while the *swing* leg is free to move above the ground. Each link is modeled as a rigid body, with both mass and rotational inertia. Links are connected to each other with ideal torque motors across frictionless revolute joints, with the exception of the ankle joint, which is passive. We have included the derivation of the equations of motion for this model in Appendix F.

**7.2. Five-Link Biped: System Dynamics.** During single stance, the five-link biped model has five degrees of freedom: the absolute angles of both lower legs ( $q_1$  and  $q_5$ ), both upper legs ( $q_2$  and  $q_4$ ), and the torso ( $q_3$ ), as shown in Figure 12. We will collect these configuration variables into the single vector  $\mathbf{q}$ . Because the model has second-order dynamics, we must also keep track of the derivative of the configuration,  $\dot{\mathbf{q}}$ . Thus, we can write the state and the dynamics as shown below, where  $\ddot{\mathbf{q}}$  is calculated from the system dynamics:

$$\mathbf{x} = \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \end{bmatrix}, \quad \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{\mathbf{q}} \\ \ddot{\mathbf{q}} \end{bmatrix}.$$

Unlike the cart-pole system, the dynamics function  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$  cannot be easily written in closed form. We show one method for deriving and evaluating the system dynamics in Appendix F.

**7.3. Five-Link Biped: Objective Function.** Just like in the cart-pole example, we will use the integral of the torque-squared cost function. This cost function tends to produce smooth, well-behaved solutions. This is desired for a few reasons. First, a smooth solution means that a piecewise polynomial spline will do a good job of approximating the solution, thus the nonlinear program will converge well. The second reason is that a smooth solution is easier to control on a real robotic system. Finally, minimizing the torque-squared tends to keep the solution away from large torques, which are sometimes undesirable in real robotic systems:

$$(7.1) \quad J = \int_0^T \left( \sum_{i=1}^5 u_i^2(\tau) \right) d\tau.$$

There are many other cost functions that we could have used. One common function is *cost of transport* (CoT), the ratio of energy used over the trajectory to the horizontal distance moved by the robot [59, 8]. It turns out that CoT is a difficult

cost function to optimize over, because the solutions tend to be discontinuous. The simple example in section 8 shows a few ways to deal with such discontinuities.

**7.4. Five-Link Biped: Constraints.** A variety of constraints are required to produce a sensible walking gait. The constraints presented here are similar to those used in [66].

First, we will require that the walking gait is *periodic*; that is, the initial state must be identical to the final state after it is mapped through heel-strike. *Heel-strike* is the event that occurs when the swing foot strikes the ground at the end of each step, becoming the new stance foot. For a single step, let's define  $\mathbf{x}_0$  to be the initial state and  $\mathbf{x}_F$  to be the final state on the trajectory, immediately *before* heel-strike. Then we can express the periodic walking constraint as shown below, where  $\mathbf{f}_H(\cdot)$  is the heel-strike map, as defined in Appendix F:

$$(7.2) \quad \mathbf{x}_0 = \mathbf{f}_H(\mathbf{x}_F).$$

Next, we would like the biped to walk at some desired speed. There are many ways to do this, but what we have chosen here is to prescribe the duration of a single step ( $T$ ) and then put an equality constraint on step length ( $D$ ). Additionally, we assume that the robot is walking on flat ground. This constraint can then be written as shown below, where  $\mathbf{P}_5(T)$  is the position of the swing foot at the end of the step and  $\mathbf{P}_0(t)$  is the position of the stance foot throughout the step. Note that we use the  $\begin{bmatrix} \cdot \\ \cdot \end{bmatrix}$  notation to show a two element column vector, where the top element is the horizontal component and the bottom element is the vertical component:

$$(7.3) \quad \mathbf{P}_5(T) = \begin{bmatrix} D \\ 0 \end{bmatrix}. \quad (\text{Note: } \mathbf{P}_0(t) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \text{ by definition.})$$

We have added an additional constraint on the biped robot to make the problem more interesting: that the stance ankle torque is identically zero throughout the trajectory. This constraint is essentially like saying “the robot has small feet,” and is widely used in the Hybrid Zero Dynamics technique for controlling walking robots [66].

When we derived the heel-strike collision equations (see Appendix F), we assumed that the trailing foot leaves the ground at the instant the leading foot collides with the ground. We can ensure that this is true by introducing a constraint that the vertical component of the swing foot velocity at the beginning of the trajectory must be positive (foot lifting off the ground) and that it must be negative at the end of the trajectory (foot moving toward the ground). These constraints can be expressed as inequality constraints on the initial and final states, where  $\hat{\mathbf{n}}$  is the normal vector of the ground. In our case,  $\hat{\mathbf{n}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ , because the ground is flat and level:

$$(7.4) \quad 0 < \dot{\mathbf{P}}_5(0) \cdot \hat{\mathbf{n}}, \quad 0 > \dot{\mathbf{P}}_5(T) \cdot \hat{\mathbf{n}}.$$

Next we have a constraint to keep the swing foot above the ground at all times, shown below. Interestingly, the optimal solution for the minimum torque-squared walking gait keeps the foot above the ground (at least for our chosen set of parameters), so this constraint is unnecessary:

$$(7.5) \quad 0 < \mathbf{P}_5(t) \cdot \hat{\mathbf{n}} \quad \forall t \in (0, T).$$

In some cases, it might be desirable to achieve some ground clearance for the swing foot, or to work with some nonflat ground profile. There are a few ways to do this.

The easiest is to require that the swing foot remain above some continuous function  $y(t)$  of time. A slightly more complicated version is to prescribe some continuous function  $y(x)$  that the swing foot must remain above, such as a simple quadratic or cubic polynomial. In both cases, it is critical that the constraint is consistent with the boundary conditions and that the implementation is smooth, to avoid over constraining the problem. Both methods are shown below, where  $\hat{\mathbf{i}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$  and  $\hat{\mathbf{j}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ :

$$(7.6) \quad y(t) < \mathbf{P}_5(t) \cdot \hat{\mathbf{n}} \quad \forall t \in (0, T), \quad \text{foot clearance (time-based),}$$

$$(7.7) \quad y(\mathbf{P}_5(t) \cdot \hat{\mathbf{i}}) < \mathbf{P}_5(t) \cdot \hat{\mathbf{j}} \quad \forall t \in (0, T), \quad \text{foot clearance (state-based).}$$

Finally, it is worth noting one mistake that is common in these optimizations: redundant constraints. Notice, for example, that for step length we only put a constraint on the final position of the foot (7.3). The initial position is fully constrained given (7.3) and the periodic step map constraint (7.2). If we were to add a constraint on the initial position of the foot, it would only serve to cause numerical problems in the nonlinear program.

**7.5. Five-Link Biped: Initialization.** When we solve the trajectory optimization problem, we need to provide an initial guess for the trajectory. In this case, we created this guess by constructing an initial and a final state, and then using linear interpolation to obtain intermediate states. We constructed the final state by selecting joint angles that formed a reasonable walking pose. We then computed the initial joint angles by applying the step map (see F.9) to the final state:

$$(7.8) \quad \mathbf{q}(0)_{\text{guess}} = \begin{bmatrix} -0.3 \\ 0.7 \\ 0.0 \\ -0.5 \\ -0.6 \end{bmatrix}, \quad \mathbf{q}(T)_{\text{guess}} = \begin{bmatrix} -0.6 \\ -0.5 \\ 0.0 \\ 0.7 \\ -0.3 \end{bmatrix},$$

$$(7.9) \quad \mathbf{q}_{\text{guess}}(t) = \mathbf{q}_{\text{guess}}(0) + \frac{t}{T}(\mathbf{q}_{\text{guess}}(T) - \mathbf{q}_{\text{guess}}(0)).$$

We initialized the joint rates by differentiating the joint angle guess:

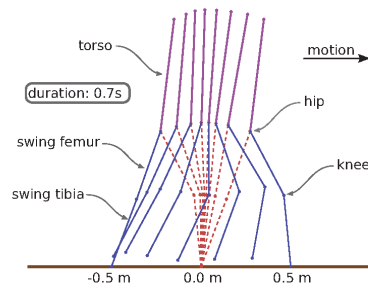
$$(7.10) \quad \dot{\mathbf{q}}_{\text{guess}}(t) = \frac{d}{dt}(\mathbf{q}_{\text{guess}}(t)) = \frac{1}{T}(\mathbf{q}_{\text{guess}}(T) - \mathbf{q}_{\text{guess}}(0)).$$

Finally, we initialized the joint torques to be constant at zero:

$$(7.11) \quad \mathbf{u}_{\text{guess}}(t) = \mathbf{0}.$$

Note that this initial guess does not satisfy the system dynamics (or most of the other constraints), but it does provide something that is close to the desired walking motion. This is the key feature of an initial guess—that it starts the optimization close enough to the desired behavior that the optimization will find the “correct” solution.

**7.6. Five-Link Biped: Results.** We solved this example problem in MATLAB, using FMINCON’s [37] interior-point algorithm as the nonlinear programming solver. The physical parameters used are given in Appendix E.2, and the optimization was



**Fig. 13** Illustration of the optimal trajectory for the five-link biped example. The poses are uniformly spaced in time and the biped is moving from left to right.

computed on a regular desktop computer.<sup>2</sup> We chose to use analytic gradients (Appendix F) for the entire problem, although similar results are obtained for numerical gradients.

All source code for solving this trajectory optimization problem, including derivation of the equations of motions, is given in the electronic supplement (see Appendix A).

We solved the problem on two meshes, using Hermite–Simpson collocation in both cases. The initial mesh had 5 segments and a low convergence tolerance (in FMINCON, ‘TolFun’ =  $1e-3$ ). For the second (final) mesh, we used a mesh with 25 segments and increased the convergence tolerance in FMINCON to ‘TolFun’ =  $1e-6$ . Both meshes had segments of uniform duration. This process could be repeated further, to achieve increasingly accurate solutions.

The solution on the initial (5-segment) mesh took 0.96 seconds to compute and 29 iterations in FMINCON’s interior-point method. The solution on the final (25-segment) mesh took 21.3 seconds to compute and 56 iterations in the NLP solver.

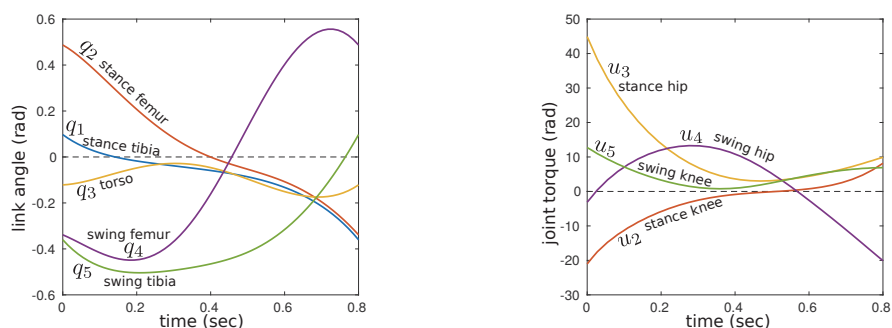
As an aside, if we solve the problem using FMINCON’s built-in numerical derivatives, rather than analytic derivatives, we obtain the same solution as before, but it takes longer: 4.30 seconds and 29 iterations on the coarse mesh, and 79.8 seconds and 62 iterations on the fine mesh. Also, for this problem, it turns out that solving on two different meshes is not critical; we could directly solve the problem on the fine (25-segment) mesh and obtain similar results.

The solution for a single periodic walking step is shown in Figure 13 as a stop-action animation with uniformly spaced frames. The same trajectory is also shown in Figure 14, with each joint angle and torque given as a continuous function of time. Finally, Figure 15 shows the error estimates computed along the trajectory.

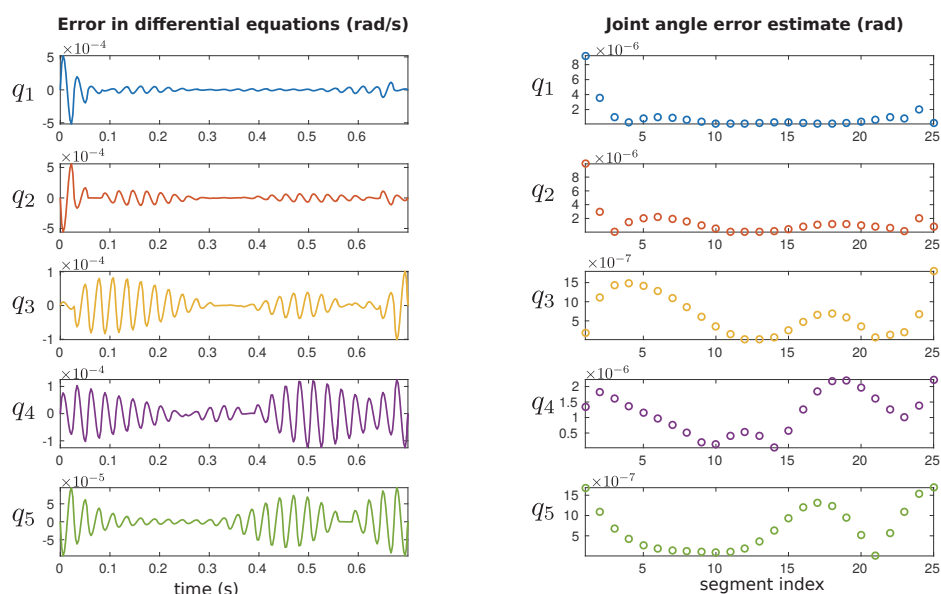
**8. Block-Move Example (Minimum-Work).** In this section, we will revisit the simple block-moving example from section 2 with a more challenging objective function. All other details of the problem remain unchanged: the block must move between two points that are one unit of distance apart in one unit of time, starting and finishing at rest. The new objective function is to minimize the integral of the absolute value of the work done by the force acting on the block.

It turns out that there is a simple analytic solution to this problem: apply maximum force to get the block up to speed, then let the block coast, then apply maximum negative force to bring it to a stop at the target point. This type of solution, which

<sup>2</sup>Processor: 3.4GHz quad-core Intel i5-3570K.



**Fig. 14** Plots showing the optimal trajectory for the five-link biped example. Notice that the curves are smooth, partially due to the integral of torque-squared cost function. The torque curve for the stance ankle  $u_1 = 0$  is not shown, because it is zero by definition.



**Fig. 15** Plots showing the error in the system dynamics along the optimal trajectory for the five-link biped example. These error estimates are computed using the techniques described in section 5.3.

consists of alternating periods of maximum and zero control effort, is known as a *bang-bang* solution. Bang-bang solutions are difficult to handle with standard direct collocation because the discretization method (based on polynomial splines) cannot accurately approximate the discontinuity in the solution. In this section, we will study a few commonly used techniques for dealing with such discontinuities in the solution to a trajectory optimization problem.

**8.1. Block-Move Example: Problem Statement.** Our goal here is to move a block one unit along a one-dimensional frictionless surface, in one unit of time, along a trajectory that minimizes the integral of the absolute work done by the control force

$u$ . The objective function is given below, where the position and velocity of the block are given by  $x$  and  $\nu$ , respectively:

$$(8.1) \quad \min_{u(t), x_1(t), \nu(t)} \int_0^1 |u(\tau) \nu(\tau)| d\tau.$$

We will assume that the block has unit mass and slides without friction, so we can write its dynamics as

$$(8.2) \quad \dot{x} = \nu, \quad \dot{\nu} = u.$$

Next, the block must start at the origin and move one unit of distance in one unit of time. Note that the block must be stationary at both start and finish:

$$(8.3) \quad \begin{aligned} x(0) &= 0, & x(1) &= 1, \\ \nu(0) &= 0, & \nu(1) &= 0. \end{aligned}$$

Finally, we will assume that the force moving the block is bounded:

$$(8.4) \quad -u_{\max} \leq u(t) \leq u_{\max}.$$

**8.2. Block-Move Example: Analytic Solution.** The analytic solution to this problem can be constructed using a slightly modified version of the method shown in Appendix B, but constraints on the control and the nonlinear objective function in this problem make the resulting formulation somewhat complicated. Instead, we will use simple intuition to make a guess at the form of the analytic solution. We find that the numerical results converge to this analytic solution, which suggests (but does not prove) that it is the correct solution.

We start by observing that in the case where  $u_{\max} \rightarrow \infty$  there is a feasible solution with zero cost: the control is a delta function at the boundaries (positive at the beginning, negative at the end) and zero otherwise. We can then extend this solution to nonzero values of  $u_{\max}$  by using a bang-bang control law: maximum force, then zero force, then minimum force. This leaves two unknowns in the control trajectory: the two switching times, which can be solved for using the boundary values for the problem. The resulting controller is

$$(8.5) \quad u^*(t) = \begin{cases} u_{\max}, & t < t^*, \\ 0 & \text{otherwise,} \\ -u_{\max}, & (1 - t^*) < t, \end{cases} \quad \text{where} \quad t^* = \frac{1}{2} \left( 1 - \sqrt{1 - \frac{4}{u_{\max}}} \right).$$

The most important aspect of this solution is that the optimal control  $u^*(t)$  is discontinuous. This means that the linear and quadratic spline control approximations used by the trapezoidal and Hermite–Simpson collocation methods *cannot* perfectly represent this solution, although they can become arbitrarily close with enough mesh refinement. One way to obtain a more precise solution would be to pose this problem as a multiphase trajectory optimization problem [45]. These methods are briefly discussed in section 9.9 and amount to solving the problem as a sequence of three coupled trajectories, allowing the discontinuity to occur precisely at the switching points between trajectories.

Another interesting point is that if  $u_{\max} < 4$ , then there is no feasible solution for the trajectory: the switching time  $t^*$  is imaginary. Finally, if there is no force limit  $u_{\max} \rightarrow \infty$ , then the solution is impulsive: not just discontinuous, but a delta function.

**8.3. Block-Move Example: Discontinuities.** There are two types of discontinuities present in this example problem. The first is obvious: the  $\text{abs}()$  in the objective function (8.1). The second discontinuity is found in the solution (8.5) itself.

There are two ways to handle the discontinuity in the objective function, both of which we will cover here. The first is to rewrite the  $\text{abs}()$  using slack variables, thus pushing the discontinuity to a set of constraints, which are easily handled by the nonlinear programming solver. The second is to replace the  $\text{abs}()$  with a smooth approximation. Both methods work, although they have different implications for the convergence time and solution accuracy, as will be demonstrated in section 8.7.

The discontinuity in the solution is a bit harder to detect and address. We can detect the discontinuity by observing that the optimization is slow to converge, and by visually inspecting the resulting trajectories. If you're stuck using single-phase direct collocation, like the methods presented in this paper, then the best way to handle the discontinuity is to smooth the problem (if possible) and then to use mesh refinement to make a dense collocation grid near the discontinuity. If you have access to a multiphase solver (see section 9.9), then you can break the trajectory into multiple segments and force the discontinuity to occur between the segments.

**8.4. Block-Move Example: Initialization.** We will compute an initial guess for position by linear interpolation between the initial position  $x(0) = 0$  and final position  $x(1) = 1$ . We then set the velocity guess to be the derivative of position and the force (acceleration) to be the derivative of velocity. There are many other schemes that could be used, but we choose this one because it is simple and effective. Once we have an initial trajectory, we can evaluate it at each collocation point to obtain values to pass to the nonlinear programming solver:

$$(8.6) \quad x^{\text{init}}(t) = t,$$

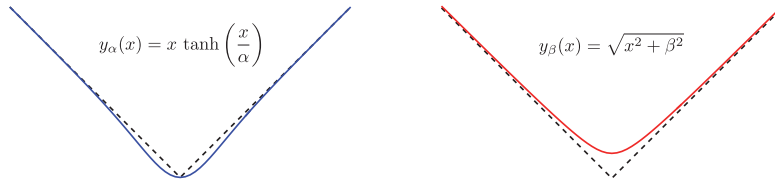
$$(8.7) \quad \nu^{\text{init}}(t) = \frac{d}{dt}x^{\text{init}}(t) = 1,$$

$$(8.8) \quad u^{\text{init}}(t) = \frac{d}{dt}\nu^{\text{init}}(t) = 0.$$

**8.5. Block-Move Example: Slack Variables.** The most “correct” way to rewrite the objective function (8.1) is using slack variables: this moves the discontinuity from the objective function to a set of constraints. The slack variable approach here is taken from [6]. The benefit of rewriting the trajectory optimization problem using slack variables to represent the absolute value function is that it is mathematically identical to the original optimization problem. That being said, there are a few downsides to this method. The first is that the solution will still be discontinuous, and direct collocation cannot precisely represent it (although it can get arbitrarily close). Second, the addition of slack variables will greatly increase the size of the nonlinear program: two additional controls and three additional constraints at every collocation point, for each  $\text{abs}()$ . Finally, the slack variables are implemented using a path constraint, which tends to cause the nonlinear program to converge more slowly.

The key idea behind the slack variable approach is that you can push the discontinuity from the objective function to a set of constraints, where the nonlinear programming solver can properly handle it. We start by introducing two slack variables ( $s_1$  and  $s_2$ ) and rewriting the objective function. Note that the slack variables





**Fig. 16** Comparison of two smooth approximations for the absolute value function: hyperbolic tangent smoothing (left) and square-root smoothing (right).

here are to be treated as decision variables for the purposes of transcription:

$$(8.9) \quad \min_{u(t), x(t), v(t)} \int_0^1 |u(t) v(t)| d\tau \quad \rightarrow \quad \min_{\substack{u(t), x(t), v(t) \\ s_1(t), s_2(t)}} \int_0^1 (s_1(\tau) + s_2(\tau)) d\tau.$$

Next, we introduce a few constraints. The first require that the slack variables be positive:

$$(8.10) \quad 0 \leq s_1(t), \quad 0 \leq s_2(t).$$

Finally, we require that the difference between the slack variables is equal to the term inside the  $\text{abs}()$  function (8.1):

$$(8.11) \quad s_1(t) - s_2(t) = u(t) v(t).$$

The set of constraints (8.10) and (8.11) means that  $s_1(t)$  represents the positive part of the argument to the  $\text{abs}()$  function, while  $s_2(t)$  represents the magnitude of the negative part.

The system dynamics, boundary constraints, and force limits remain unchanged. This modified version of the problem is now acceptable to pass into a nonlinear programming solver. There are many possible ways to initialize the slack variables, but we've found that  $s_1(t) = s_2(t) = 0$  is a good place to start.

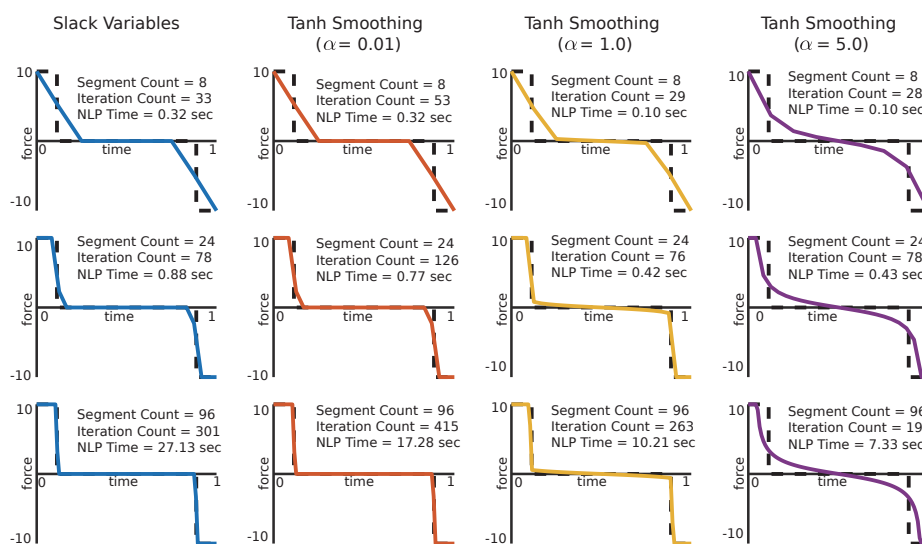
The resulting nonlinear program does not solve quickly, but the solver will eventually find a solution. The result will be the best possible trajectory, given the limitations caused by the spline approximation in the transcription method, as shown in section 8.7.

**8.6. Block-Move Example: Smoothing.** Although the slack variable method for representing  $\text{abs}()$  is exact, the resulting nonlinear program can be complicated to construct and slow to solve. An alternative approach is to replace the  $\text{abs}()$  function with a smooth approximation. This method is simple to implement and solve, but at a loss of accuracy. Here we will discuss two smooth approximations for  $\text{abs}()$ , both of which are given below and plotted in Figure 16:

$$(8.12) \quad y_\alpha(x) = x \tanh\left(\frac{x}{\alpha}\right) \approx |x|,$$

$$(8.13) \quad y_\beta(x) = \sqrt{x^2 + \beta^2} \approx |x|,$$

The smooth approximation to  $\text{abs}()$  using the hyperbolic tangent function (8.12), also known as exponential smoothing, is always less than  $|x|$ , while the approximation using the square-root function (8.13) is always greater than  $|x|$ . The smoothing



**Fig. 17** Plots showing the solution to the minimal-work block-moving example, computed using various methods and parameters. In each case, the analytic solution is given by a dashed black line, and the solid colored line gives the numerical solution using direct collocation. The left column shows the solution when the  $\text{abs}()$  in the objective function is handled with slack variables. The remaining columns show the result obtained using  $\tanh()$  smoothing, for light smoothing ( $\alpha = 0.01$ ), medium smoothing ( $\alpha = 1.0$ ), and heavy smoothing ( $\alpha = 5.0$ ). Notice that the solution obtained using slack variables and light smoothing are similar to one another, with the smoothing taking more iterations but less time. The problem solves even faster with medium and heavy smoothing, although the solution accuracy is degraded. Note that the smoothed version of the problem results in a more smooth solution.

parameters  $\alpha$  and  $\beta$  can be used to adjust the amount of smoothing on the problem, with the smooth versions of the functions approaching  $|x|$  as  $\alpha \rightarrow 0$  and  $\beta \rightarrow 0$ . The size of these smoothing parameters and choice of smoothing method are both problem dependent. In general, smaller values for the smoothing parameters make the nonlinear program increasingly difficult to solve, but with a more accurate solution.

One important thing to note is that smoothing fundamentally changes the optimization problem, and not necessarily in an obvious way. For this reason, it is important to do convergence tests, solving the problem with successively smaller and smaller values for the smoothing parameter to ensure the correct solution is obtained. An example of this can be found in both [55] and [9].

**8.7. Block-Move Example: Results.** We solved this more complicated version of the block-moving problem using the trapezoidal collocation method, and we used FMINCON's [37] interior-point solver to solve the nonlinear program. Although this optimization problem appears simple, it is actually difficult to solve numerically without careful mesh refinement (or reposing the problem using multiphase trajectory optimization; see section 9.9). To illustrate some trade-offs, we have solved the problem on three different meshes, using both slack variables and smoothing to handle the  $\text{abs}()$  function in the objective. Figure 17 shows the solution for each of these different setups and compares each one to the analytic solution. All solutions were obtained using the same solver settings and initialization, and the source code is included in the electronic supplement (see Appendix A).

One interesting thing to notice is that all of these solutions require a large number of iterations to solve the nonlinear program, compared to both the cart-pole swing-up problem and the five-link biped problem. This might seem odd, since this block-pushing problem looks like it should be easier. The difficulty, as best we can tell, comes from the discontinuity in the solution.

The solution obtained using slack variables (left column) converges to the analytic solution, although it takes some time and a very fine mesh. The solution using light smoothing ( $\alpha = 0.01$ ) is quite close to the solution obtained with slack variables, although the smooth version of the problem takes more iterations (because the problem is stiff) and less time (because of the smaller number of decision variables). As the smoothing parameter is increased ( $\alpha = 1.0$  and  $\alpha = 5.0$ ), the solution is obtained faster, at a loss of accuracy.

**9. Background.** The topics in this section are selected to provide the reader with a broad understanding of some of the concepts that are related to direct collocation. We start with a few topics about optimization in general and then move on to other methods for solving trajectory optimization problems. We conclude with a method comparison and a list of optimization software.

**9.1. Trajectory Optimization vs. Parameter Optimization.** Trajectory optimization is concerned with minimizing a *functional*  $J(\mathbf{f}(t))$ , where  $\mathbf{f}(t)$  is an arbitrary vector function. In contrast, parameter optimization is concerned with minimizing some function  $J(\mathbf{x})$ , where  $\mathbf{x}$  is a vector of real numbers. This makes trajectory optimization more challenging than parameter optimization, because the space of functions is much larger than the space of real numbers.

**9.2. Open-Loop vs. Closed-Loop Solutions.** Trajectory optimization is a collection of techniques that are used to find *open-loop* solutions to an optimal control problem. In other words, the solution to a trajectory optimization problem is a sequence of controls  $\mathbf{u}^*(t)$ , given as a function of time, that moves a system from a single initial state to some final state. This sequence of controls, combined with the initial state, can then be used to define a single trajectory that the system takes through state space.

There is another set of techniques, known as *dynamic programming*, which find an optimal *policy*. Unlike an optimal trajectory, an optimal policy provides the optimal control for *every* point in the state space. Another name for the optimal policy is the *closed-loop* solution to the optimal control problem. An optimal trajectory starting from any point in the state space can be recovered from a closed-loop solution by a simple simulation. Figure 18 illustrates the difference between an open-loop and a closed-loop solution.

In general, trajectory optimization is most useful for systems that are high-dimensional, have a large state space, or need to be very accurate. The resulting solution is open-loop, so it must be combined with a stabilizing controller when applied to a real system. One major shortcoming of trajectory optimization is that it will sometimes fail to converge, or will converge to a *locally optimal* solution, failing to find the *globally optimal* solution.

Dynamic programming (computing an optimal policy) tends to be most useful on lower-dimensional systems with small but complex state spaces, although some variants have been applied to high-dimensional problems [46]. There are two advantages to dynamic programming over trajectory optimization. The first is that dynamic programming gives the optimal control for *every* point in state space, and can thus be



**Fig. 18** Comparison of an open-loop solution (optimal trajectory) with a closed-loop solution (optimal policy). An open-loop solution (left) to an optimal control problem is a sequence of controls  $u(t)$  that move the system from a single starting point **A** to the destination point **B**. In contrast, the closed-loop solution gives the controls  $u(x)$  that can move the system from any point in the state space to the destination point **B**.

applied directly to a real system. The second and perhaps more important advantage is that it will (at least in the basic formulations) always find the globally optimal solution. The downside of dynamic programming is that computing the optimal solution for every point in the state space is very expensive, scaling exponentially with the dimension of the problem—the so-called curse of dimensionality [42].

**9.3. Continuous-Time and Discrete-Time Systems.** Trajectory optimization is generally concerned with finding optimal trajectories for a *dynamical system*. The dynamics describe how the state of a system changes in response to some input or decision, typically referred to as a control.

There are many different types of dynamical systems. In this tutorial we have focused on *continuous-time* dynamical systems, which have continuous time, state, and control. This type of system is common in robotics and the aerospace industry, for example, when planning the trajectory that a spacecraft will take between two planets:

$$(9.1) \quad \dot{x} = f(t, x, u), \quad \text{continuous-time system.}$$

Another common system is a *discrete-time* dynamical system, which has discrete time steps, but continuous state and control. This type of system is commonly used in model predictive control, for example, in building climate control systems [36]. Trajectory optimization for these systems is generally easier than for fully continuous systems. Discrete-time systems are often constructed to approximate continuous time systems:

$$(9.2) \quad x_{k+1} = f_k(x_k, u_k), \quad \text{discrete-time system.}$$

A final type of dynamical system is a *directed graph*, where there is a finite set of states (nodes on the graph) and controls (transitions, actions, edges on the graph). Most algorithms for computing an optimal policy (optimal control from every point in the state space) require the dynamical system to be in this discrete form. A common example would be a traffic network, where there is a discrete set of states (cities) and a discrete set of controls (roads out of each city). Sometimes continuous-time problems are abstracted into this form so that they can make use of sophisticated graph search algorithms to approximate the optimal policy.

**9.4. Indirect Methods.** Both the trapezoidal and Hermite–Simpson collocation methods presented in this tutorial are *direct* methods, which discretize the trajectory

optimization problem, converting it into a nonlinear program. There is another set of methods for solving trajectory optimization problems, known as *indirect* methods. Indirect methods analytically construct the necessary and sufficient conditions for optimality, then they discretize these conditions and solve them numerically. A common way to distinguish these two methods is that a direct method discretizes and then optimizes, while an indirect method optimizes and then discretizes.

Let's consider a simple scalar optimization problem to illustrate how an indirect method works: minimizing  $y = f(t)$ . Basic calculus tells us that the minimum value  $y^* = f(t^*)$  will occur when the derivative is zero,  $y'(t^*) = 0$ . Additionally, we need to check that the curvature is positive,  $y''(t^*) > 0$ , ensuring that we have a local minimum, rather than a local maximum (or saddle point). If both of those conditions hold, then we know that  $y^* = f(t^*)$  is indeed a local minimum. An indirect optimization works along the same principle, but the conditions are a bit more difficult to construct and solve. In contrast, a direct method will minimize  $y(t)$  by constructing a sequence of guesses such that each subsequent guess is an improvement on the previous one:  $y(t_0) > y(t_1) > \dots > y(t^*)$  [6].

The major benefit of an indirect method, when compared to a direct method, is that an indirect method will generally be more accurate and will have a more reliable error estimate. Both of these benefits come from the analytic expressions for the necessary and sufficient conditions that the user derives while constructing the indirect problem.

There are several difficulties associated with indirect methods when compared to direct methods. For example, the region of convergence tends to be smaller for indirect methods than direct methods, which means that an indirect method will require a better initialization [5]. Furthermore, the initialization of an indirect method is complicated by the need to initialize the adjoint variables, which are not used in a direct method [6]. Finally, in order to obtain an accurate solution for an indirect method, it is typically necessary to construct the necessary and sufficient conditions analytically, which can be challenging [5].

**9.5. Direct Single Shooting.** Like direct collocation, the *direct single shooting method* (also known as *single shooting*) solves a trajectory optimization problem by transforming it into a nonlinear program. The key difference is that a direct shooting method approximates the trajectory using a simulation. The decision variables in the nonlinear program are an open-loop parameterization of the control along the trajectory, as well as the initial state. Direct shooting is well suited to applications where the control is simple and there are few path constraints, such as space flight [5].

**9.6. Direct Multiple Shooting.** A common extension of the direct single shooting method is *direct multiple shooting* (also called *parallel shooting*). Rather than representing the entire trajectory as a single simulation, the trajectory is divided up into segments, and each segment is represented by a simulation. Multiple shooting tends to be much more robust than single shooting, and thus is used on more challenging trajectory optimization problems [5].

When compared to collocation methods, shooting methods tend to create small dense nonlinear programs, which have fewer decision variables that are more coupled. One difficulty with direct shooting methods is that it is difficult to implement path constraints, since the intermediate state variables are not decision variables in the nonlinear program [5]. Another difficulty with shooting methods, particularly with direct shooting, is that the relationship between the decision variables and constraints is often highly nonlinear, which can cause poor convergence in some cases [5, 6].

**9.7. Orthogonal Collocation.** Orthogonal collocation is similar to direct collocation, but it generally uses higher-order polynomials. The collocation points for these methods are located at the roots of an orthogonal polynomial, typically either Chebyshev or Legendre [15]. Increasing the accuracy of a solution is typically achieved by increasing either the number of trajectory segments or the order of the polynomial in each segment.

One important reason to use high-order orthogonal polynomials for function approximation is that they achieve *spectral* convergence. This means that the convergence rate is exponential in the order of the polynomial [51], *if* the underlying function is sufficiently smooth [58]. In cases where the entire trajectory is approximated using a single high-order polynomial, the resulting method is called *pseudospectral* collocation or *global collocation* [51].

One of the key implementation details about orthogonal collocation is that the trajectory is represented using *barycentric interpolation* [4], rather than directly from the definition of the orthogonal polynomial. Barycentric interpolation provides a numerically efficient and stable method for interpolation, differentiation, and quadrature, all of which can be computed by knowing the trajectory's value at the collocation points. See Appendix D for further details about how to work with orthogonal polynomials.

**9.8. Differential Dynamic Programming.** One final method is *differential dynamic programming*. It is similar to direct shooting, in that it simulates the system forward in time and then optimizes based on the result of that simulation. The difference is in how the optimization is carried out. While direct shooting uses a general-purpose nonlinear programming solver, the differential dynamic programming algorithm optimizes the trajectory by propagating the optimal control backward along the candidate trajectory. In other words, it exploits the time-dependent nature of the trajectory. It was described in [39, 31], and a good overview was provided by [41].

**9.9. Multiphase Methods.** There are many trajectory optimization problems that have a sequence of continuous-motion phases separated by discrete jumps. One common example is the trajectory of a multistage rocket, which has continuous motion punctuated by discrete changes when each stage separates. Another example is the gait of a walking robot, which has a discontinuity as each foot strikes the ground. Solving a multiphase problem is sort of like solving multiple single-phase problems in parallel. The key difference is that the boundary constraints between any two phases can be connected, thus coupling the trajectory segments. Multiphase methods are covered in detail in [45, 63].

**9.10. Through-Contact Methods.** Through-contact methods are specialized for computing optimal trajectories for hybrid dynamical systems that describe contact mechanics: imagine the gait of a walking robot, or two objects colliding and then falling to the ground. Most physics simulators use a complementarity constraint to model contact between two rigid objects: a contact force is allowed if and only if the two objects are in contact. The key idea in through-contact optimization is to treat the contact forces as decision variables in the optimization, and then apply a complementarity constraint at each grid point: the contact force must be zero unless the objects are in contact. These methods are covered in detail in [48], [47], and [40].

**9.11. Which Method Is Best?** In short, there is no best method for trajectory optimization. There are many trade-offs between the different methods, and a good understanding of these trade-offs will help determine which method is best for a

specific application. A good high-level comparison of methods can be found in [5] and [51]. Here I will provide a brief overview of some of these trade-offs.

In general, indirect methods tend to produce more accurate solutions than direct methods, at the cost of being more difficult to construct and solve. This is because indirect methods explicitly compute the necessary and sufficient conditions for optimality of the original problem, while a direct method precisely solves a discrete approximation of the original problem. One common approach to obtaining accurate solutions is to first compute an approximation of the solution using a direct method, and then use this to initialize an indirect method. As a side note: both shooting and collocation (transcription) methods can be applied to either a direct or an indirect formulation of a trajectory optimization problem [6].

Shooting methods are best for applications where the dynamics must be computed accurately, but the control trajectory is simple, for example, in computing the trajectory of a spacecraft, where you occasionally fire the thrusters to change course, but otherwise follow a ballistic trajectory. Multiple shooting methods are generally preferred over single shooting, except in cases where the control is very simple or the initial guess is very good.

Collocation (transcription) methods are best for applications where the dynamics and control must be computed to a similar accuracy, and the structure of the control trajectory is not known a priori, for example, in computing the torque to send to the joints of a robot as it performs some motion.

Both shooting and collocation methods can be either low or high order. High-order collocation methods are given a special name: orthogonal collocation. Trapezoidal collocation would be considered a low-order method, while Hermite–Simpson collocation would usually be considered a medium-order method. The trade-off between using a method with more low-order segments or one with fewer high-order segments is complicated [16]. The general approach is to use a relatively lower-order method to obtain an initial solution to the trajectory and then perform an error analysis [6, 16]. The result will indicate whether it is better to remesh the trajectory using additional lower-order segments or to replace lower-order segments with higher-order segments.

In situations where you need to compute the trajectory for a hybrid system, there are two choices: multiphase optimization (section 9.9) and through-contact optimization (section 9.10). Multiphase optimization is preferable for most situations: the optimizations are easier to compute and tend to be more accurate. Through-contact optimization is preferable when the discontinuities are due to contact mechanics and the sequence of continuous-motion phases is unknown.

**9.12. Trajectory Optimization Software.** There are a variety of software programs that solve trajectory optimization problems, some of which are given in Table 1. Each of these solvers performs some transcription method and then hands the problem off to a nonlinear programming solver.

Table 2 shows a few popular software packages for solving nonlinear programming problems.

The electronic supplement, described in Appendix A, also includes a MATLAB library for trajectory optimization. It was written to accompany this tutorial, and it implements trapezoidal and Hermite–Simpson collocation, as well as all four example problems.

**10. Summary.** The goal of this tutorial is to give the reader an understanding of the concepts required to implement their own direct collocation methods. We focus

**Table 1** *Trajectory optimization software.*

| Name          | License      | Interface | Method                                           |
|---------------|--------------|-----------|--------------------------------------------------|
| GPOPS-II [45] | commercial   | MATLAB    | direct orthogonal collocation                    |
| PSOPT [2]     | open source  | C++       | direct collocation                               |
| SOS [7]       | commercial   | GUI       | direct collocation (methods from [6])            |
| DIRCOL [63]   | free license | C         | direct collocation                               |
| DIDO [52]     | commercial   | MATLAB    | indirect orthogonal (pseudospectral) collocation |

**Table 2** *Nonlinear programming solvers.*

| Name                | License             | Interface |
|---------------------|---------------------|-----------|
| <i>FMINCON</i> [37] | MATLAB (commercial) | MATLAB    |
| <i>SNOPT</i> [25]   | commercial          | C++       |
| <i>IPOPT</i> [64]   | open source         | C++       |

primarily on trapezoidal and Hermite–Simpson collocation, and we briefly touch on a variety of other methods. We include practical suggestions, debugging techniques, and a complete set of equations and derivations. Throughout the tutorial we convey concepts through a sequence of four example problems, and the electronic supplement shows how to solve each example using MATLAB.

**Appendix A. Overview of Electronic Supplementary Material.** This tutorial has an accompanying electronic supplement that contains two parts. The first part is a general-purpose trajectory optimization library, written in MATLAB, that solves trajectory optimization problems of the type presented here. The second part of the supplement is a set of code that solves each of the example problems in this tutorial. There are also a few other MATLAB scripts, which can be used to derive some of the equations in the text and to generate some of the simple figures.

All of the source code in the electronic supplement is well documented, with the intention of making it easy to read and understand. Each directory in the supplement contains a README file that gives a summary of the contents.

**A.1. Trajectory Optimization Code.** The electronic supplement includes a general-purpose MATLAB library for solving trajectory optimization problems, written by the author. The source code is well documented, such that it can be read as a direct supplement to this tutorial. The code is still under development, and the most up-to-date version is publicly available on GitHub:

<https://GitHub.com/MatthewPeterKelly/OptimTraj>

The trajectory optimization code allows the user to choose from four different methods: trapezoidal direct collocation, Hermite–Simpson direct collocation, fourth-order Runge–Kutta direct multiple shooting, and Chebyshev orthogonal collocation (global Lobatto method). The user can easily switch between methods and specify a mesh refinement schedule.

The solution is returned to the user at each grid point along the trajectory. In addition, a function handle is provided to compute method-consistent interpolation for each component of the solution and both direct collocation methods provide the user with an error estimate along the solution trajectory.



**A.2. Example Problems.** The electronic supplement includes a solution (in MATLAB) to each of the four examples in this tutorial. Each example is in its own directory and calls the trajectory optimization code from Appendix A.1. Some example problems are implemented with many files, but the entry-point script always has the prefix MAIN. In some cases an additional script with the prefix RESULTS is included, which is used to generate figures from the tutorial.

Both the cart-pole and five-link biped examples make use of the MATLAB symbolic toolbox to generate their equations of motion. These automatically generated files have the prefix autoGen\_ and are created by a script with the prefix Derive.

**Appendix B. Analytic Solution to Block-Move Example.** In this section we show how to find the analytic solution to the block-moving problem from section 2. The method presented here is based on the calculus of variations and is described in detail in the textbook by Bryson and Ho [11]. Here we show two slightly different solution methods. The first solution, in section B.1, treats the problem as a true optimal control problem, where the state and control are separate and the dynamics are handled with multiplier functions. The second solution, in section B.2, simplifies the problem by first substituting the dynamics into the cost function.

**B.1. Full Solution.** We would like to minimize the cost functional  $J()$ , given below, where  $u$  is the control force applied to the block,

$$(B.1) \quad J(t, \mathbf{z}, u) = \int_0^1 u^2(\tau) d\tau.$$

The system dynamics  $\mathbf{f}()$  are given below, where  $x$  is position,  $v$  is velocity, and  $\mathbf{z} = [x, v]^T$  is the state vector:

$$(B.2) \quad \dot{\mathbf{z}} = \begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \mathbf{f}(\mathbf{z}, u) = \begin{bmatrix} v \\ u \end{bmatrix}.$$

We will also apply the following boundary conditions, where subscripts are used to denote evaluation at the boundary points on the trajectory:

$$(B.3) \quad \mathbf{z}_0 = \mathbf{z}(t)|_{t=0} = \begin{bmatrix} x_0 \\ v_0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{z}_1 = \mathbf{z}(t)|_{t=1} = \begin{bmatrix} x_1 \\ v_1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

We need to satisfy the dynamics to ensure a feasible solution. This is done by modifying the cost functional to include the system dynamics and a vector of multiplier functions  $\boldsymbol{\lambda} = [\lambda_x, \lambda_v]^T$ . Notice that when the dynamics are satisfied,  $\mathbf{f} - \dot{\mathbf{z}} = 0$  and thus  $\bar{J} = J$ , regardless of what the multiplier functions are:

$$(B.4) \quad \bar{J} = \int_0^1 (u^2(\tau) + \boldsymbol{\lambda}^T (\mathbf{f} - \dot{\mathbf{z}})) d\tau.$$

Now we can use integration by parts to rewrite the modified cost function [11]. Here we again use the subscript notation to indicate evaluation at the boundary condition (e.g.,  $\boldsymbol{\lambda}_0 = \boldsymbol{\lambda}(t)|_{t=0}$ ):

$$(B.5) \quad \bar{J} = \boldsymbol{\lambda}_0^T \mathbf{z}_0 - \boldsymbol{\lambda}_1^T \mathbf{z}_1 + \int_0^1 (u^2(\tau) + \boldsymbol{\lambda}^T \mathbf{f}) + (\dot{\boldsymbol{\lambda}}^T \mathbf{z}) d\tau.$$

At this point, it is useful to define two quantities that will be useful throughout the rest of the derivation. The first is the *Lagrangian*  $\mathcal{L}$ , which is the term inside the

integral of the original cost function  $J$ . The second term is the *Hamiltonian*  $\mathcal{H}$ , which is the sum of the Lagrangian and product of the multiplier functions with the system dynamics [11]:

$$(B.6) \quad \mathcal{L} = u^2,$$

$$(B.7) \quad \mathcal{H} = \mathcal{L} + \boldsymbol{\lambda}^T \mathbf{f} = u^2 + \lambda_x \nu + \lambda_\nu u.$$

Consider a simple optimization problem: finding the minimum of a scalar function. The minimum will occur when the first derivative is zero and the second derivative is positive. A similar principle can be used for trajectories, although we use the term *variation* instead of derivative. An optimal trajectory must have a first variation equal to zero and a second variation that is nonnegative. Here we will focus on the necessary condition, that the first variation is zero.

Let's suppose that the optimal trajectory is given by  $\mathbf{z}^*$  and  $u^*$ . A trajectory that is suboptimal can now be written as a sum of the optimal trajectory and a small perturbation from that trajectory, as shown below, where  $\varepsilon$  is a small parameter and  $\delta \mathbf{z}$  and  $\delta u$  are small (arbitrary) variations in the state and control:

$$(B.8) \quad \delta \mathbf{z} = \mathbf{z}^* + \varepsilon \delta \mathbf{z}, \quad u = u^* + \varepsilon \delta u.$$

The first variation of the cost function is its partial derivative with respect to this small parameter  $\varepsilon$ :

$$(B.9) \quad \delta \bar{J} \equiv \left. \frac{\partial}{\partial \varepsilon} \bar{J} \right|_{\varepsilon=0}.$$

Using the chain rule, we can now write out an expression for the first variation of the cost function [11]:

$$(B.10) \quad \delta \bar{J} = \boldsymbol{\lambda}_0^T \left. \frac{\partial \mathbf{z}_0}{\partial \varepsilon} \right|_{\varepsilon=0} - \boldsymbol{\lambda}_1^T \left. \frac{\partial \mathbf{z}_1}{\partial \varepsilon} \right|_{\varepsilon=0} + \int_0^1 \left[ \left. \frac{\partial \mathcal{H}}{\partial \varepsilon} \right|_{\varepsilon=0} + \dot{\boldsymbol{\lambda}}^T \left. \frac{\partial \mathbf{z}}{\partial \varepsilon} \right|_{\varepsilon=0} \right] d\tau,$$

$$(B.11) \quad \delta \bar{J} = \boldsymbol{\lambda}_0^T \delta \mathbf{z}_0 - \boldsymbol{\lambda}_1^T \delta \mathbf{z}_1 + \int_0^1 \left[ \left( \frac{\partial \mathcal{H}}{\partial \mathbf{z}} + \dot{\boldsymbol{\lambda}}^T \right) \delta \mathbf{z} + \frac{\partial \mathcal{H}}{\partial \mathbf{u}} \delta \mathbf{u} \right] d\tau.$$

The first variation of the cost function  $\delta \bar{J}$  (B.11) must be zero along the optimal trajectory. The variations in state at the initial and final points on the trajectory are zero, since the boundary conditions are fixed ( $\delta \mathbf{z}_0 = 0$ ,  $\delta \mathbf{z}_1 = 0$ ). Thus the first two terms in (B.11) are both zero. The variations in state  $\delta \mathbf{z}$  and in control  $\delta \mathbf{u}$  along the trajectory are arbitrary, thus each of their coefficients must be zero in order for the integral term to be zero:

$$(B.12) \quad \frac{\delta \mathcal{H}}{\delta \mathbf{z}} + \dot{\boldsymbol{\lambda}}^T = 0,$$

$$(B.13) \quad \frac{\delta \mathcal{H}}{\delta \mathbf{u}} = 0.$$

These two equations (B.12) and (B.13) form the necessary conditions for optimality: a solution that satisfies them will be at a stationary point. To be rigorous, we would also need to show that the second variation is nonnegative, which implies that the solution is at a minimum (as opposed to a maximum or saddle point). This calculation is beyond the scope of this paper, but is covered in [11].

The next step is to solve for the multiplier functions, which we do by rearranging (B.12) to give us the differential equations

$$(B.14) \quad -\dot{\lambda}^T = \frac{\delta \mathcal{H}}{\delta z},$$

$$(B.15) \quad \dot{\lambda} = -\left(\frac{\partial \mathcal{L}}{\partial z}\right)^T - \left(\frac{\partial f}{\partial z}\right)^T \lambda.$$

We can now evaluate (B.15) for our specific problem:

$$(B.16) \quad \begin{bmatrix} \dot{\lambda}_x \\ \dot{\lambda}_\nu \end{bmatrix} = -\begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \lambda_x \\ \lambda_\nu \end{bmatrix}.$$

This system of equations (B.16) is linear, and thus a solution is easily obtained, where  $c_0$  and  $c_1$  are constants of integration and time is given by  $t$ :

$$(B.17) \quad \lambda_x = c_0,$$

$$(B.18) \quad \lambda_\nu = c_1 - c_0 t.$$

Now that we know the multiplier functions, we can go back and solve for the control functions using (B.13):

$$(B.19) \quad 0 = \frac{\partial \mathcal{H}}{\partial u},$$

$$(B.20) \quad 0 = \frac{\partial}{\partial u}(u^2 + \lambda_x \nu + \lambda_\nu u),$$

$$(B.21) \quad 0 = 2u + 0 + (c_1 - c_0 t),$$

$$(B.22) \quad u = \frac{1}{2}(c_0 t - c_1).$$

We can use the system dynamics to obtain expressions for the position and velocity as functions of time:

$$(B.23) \quad \nu = \int u(\tau) d\tau = \frac{1}{4}c_0 t^2 - \frac{1}{2}c_1 t + c_2,$$

$$(B.24) \quad x = \int \nu(\tau) d\tau = \frac{1}{12}c_0 t^3 - \frac{1}{4}c_1 t^2 + c_2 t + c_3.$$

Next, we need to solve for the unknown constants of integration  $c_i$ . We can do this by constructing a linear system from the boundary conditions:

$$(B.25) \quad \begin{bmatrix} x(0) \\ v(0) \\ x(1) \\ v(1) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ \frac{1}{12} & -\frac{1}{4} & 1 & 1 \\ \frac{1}{4} & -\frac{1}{2} & 1 & 0 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}.$$

Solving the linear system and substituting in the coefficients yields the solution below, which is valid for the domain of the problem  $t \in [0, 1]$ :

$$(B.26) \quad x(t) = -2t^3 + 3t^2,$$

$$(B.27) \quad \nu(t) = -6t^2 + 6t,$$

$$(B.28) \quad u(t) = -12t + 6.$$

**B.2. Short Solution.** For this problem, a shorter solution can be obtained since the control  $u$  is simply the second derivative of the position  $x$ . As a result, our cost function can be written as

$$(B.29) \quad J = \int_0^1 u^2(\tau) d\tau = \int_0^1 \ddot{x}^2(\tau) d\tau.$$

In this case, we obtain the Lagrangian

$$(B.30) \quad \mathcal{L}(t, x, \dot{x}, \ddot{x}) = \mathcal{L}(\ddot{x}) = \ddot{x}^2.$$

For a fully rigorous solution, one would need to show that the first variation of the objective function is zero and the second variation is nonnegative. Here we will focus on the first variation, which is the necessary condition for the optimal solution  $x^*$ . The following equation is constructed using integration by parts:

$$(B.31) \quad \frac{\partial \mathcal{L}}{\partial x^*} - \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{x}^*} - \frac{d^2}{dt^2} \frac{\partial \mathcal{L}}{\partial \ddot{x}^*} = 0.$$

The first two terms are zero, since  $\mathcal{L}$  depends only on  $\ddot{x}$ . The final term can be evaluated and simplified to arrive at the following ordinary differential equation:

$$(B.32) \quad (0) - (0) - \frac{d^2}{dt^2} (2\ddot{x}^*) = 0,$$

$$(B.33) \quad \frac{d^4}{dt^4} x^* = 0.$$

The solution to this equation is a cubic polynomial with four unknown coefficients, identical to that found in (B.24). We solve for these coefficients using the boundary conditions (B.3) to arrive at the solution

$$(B.34) \quad x(t) = -2t^3 + 3t^2.$$

**Appendix C. Derivation of Simpson Quadrature.** Simpson quadrature is used to compute an approximation to the definite integral of a function by evaluating it at the boundaries and midpoint of the domain. It is precise when this function (the integrand) is quadratic, and we will use this fact to derive the rule. Let's start with the following quadratic curve  $\nu(t)$ :

$$(C.1) \quad \nu(t) = A + Bt + Ct^2.$$

Now suppose that we wish to compute a quantity  $x$  by integrating the function  $\nu(t)$ :

$$(C.2) \quad x = \int_0^h \nu(t) dt,$$

$$(C.3) \quad x = \int_0^h A + Bt + Ct^2 dt,$$

$$(C.4) \quad x = At + \frac{1}{2}Bt^2 + \frac{1}{3}Ct^3 \Big|_0^h,$$

$$(C.5) \quad x = Ah + \frac{1}{2}Bh^2 + \frac{1}{3}Ch^3.$$

We can use the value of  $\nu$  at three points to uniquely determine the value of the coefficients  $A$ ,  $B$ , and  $C$ . We will choose these points to be at the boundaries and midpoint of the interval:

$$(C.6) \quad \nu(0) = \nu_L, \quad \nu\left(\frac{h}{2}\right) = \nu_M, \quad \nu(h) = \nu_U.$$

Doing a bit of algebra will show that the coefficients are given by

$$(C.7) \quad A = \nu_L,$$

$$(C.8) \quad Bh = -3\nu_L + 4\nu_M - \nu_U,$$

$$(C.9) \quad Ch^2 = 2\nu_L - 4\nu_M + 2\nu_U.$$

Finally, we can plug these coefficients into (C.5) and then simplify to arrive at Simpson's rule for quadrature:

$$(C.10) \quad x = \frac{h}{6}(\nu_L + 4\nu_M + \nu_U).$$

**Appendix D. Orthogonal Polynomials.** All direct collocation methods are based on using polynomial splines to approximate continuous functions. The trapezoidal and Hermite–Simpson methods that we covered in this article both use relatively low-order polynomial splines. Orthogonal collocation methods are similar, but use high-order splines instead. Working with these high-order polynomials requires some special attention to ensure that the implementations are numerically stable.

The basic idea behind function approximation with orthogonal polynomials is that any function can be represented by an infinite sum of basis functions. The Fourier series is one well-known example, where you can represent an arbitrary function by an infinite sum of sine and cosine functions. A rough approximation of the function can be made by including a small number of terms in the sum, while a more accurate approximation can be made by including more terms. It turns out that if the function of interest is smooth, as is often the case in trajectory optimization, then orthogonal polynomials make an excellent choice of basis function. The number of terms in the infinite series is related to the *order* of the polynomial: a higher-order polynomial approximation will be more accurate. There are many papers that cover the detailed mathematics of orthogonal polynomials [26, 44, 4, 33, 58, 28] and their use in trajectory optimization [29, 62, 18, 30, 19, 53, 3, 57, 23, 24, 15, 21]. Here we will focus on the practical implementation details and on gaining a qualitative understanding of how orthogonal collocation works.

For the rest of this section, let's assume that we have some function  $f(t)$  that we would like to approximate over the interval  $[-1, 1]$ . We can do this using *barycentric interpolation*: representing the function's value at any point on the interval by a convex combination of its value at several carefully chosen *interpolation (grid) points*. We will write these points as  $t_i$  and the value of the function at these points as  $f_i$ . The set of points  $t_i$  can then be used to compute a set of interpolation weights  $v_i$ , quadrature weights  $w_i$ , and a differentiation matrix  $D$ . If the points  $t_i$  are chosen to be the roots of an orthogonal polynomial, and the function  $f(t)$  is smooth, then the resulting interpolation, integration, and differentiation schemes tend to be both accurate and easy to compute. Other distributions of points  $t_i$  do not give nice results. For example, choosing  $t_i$  to be uniformly spaced over the interval will result in numerically unstable schemes [4].

Orthogonal collocation techniques for trajectory optimization make extensive use of these properties of orthogonal polynomials. In particular, the differentiation matrix can be used to construct a set of collocation constraints to enforce the dynamics of a system, the quadrature weights can be used to accurately approximate an integral cost function or constraint, and barycentric interpolation can be to evaluate the solution trajectory.

For the rest of this section we will assume that the function of interest has been mapped to the interval  $t \in [-1, 1]$ . If the function is initially defined on the interval

$\tau \in [\tau_A, \tau_B]$ , this mapping can be achieved by

$$(D.1) \quad t = 2 \frac{\tau - \tau_A}{\tau_B - \tau_A} - 1.$$

**D.1. Computing Polynomial Roots.** An orthogonal polynomial approximation can be defined by the value of the function  $f(t)$  at the roots  $t_i$  of that orthogonal polynomial. There are many different orthogonal polynomials to choose from, each of which has slightly different properties. The ChebFun [17] library for MATLAB provides subroutines for computing the interpolation points  $t_i$ , interpolation weights  $v_i$ , and quadrature weights  $w_i$  for most common orthogonal polynomials.

The Chebyshev orthogonal polynomials are one popular choice, in part because their roots are easy to compute. The Chebyshev–Lobatto points, also called the Chebyshev points of the second kind, are given by [58] and shown below:

$$(D.2) \quad t_i = \cos\left(\frac{i\pi}{n}\right), \quad 0 \leq i \leq n.$$

The Legendre orthogonal polynomials are also commonly used. Unlike the Chebyshev polynomials, the roots of the Legendre polynomials have no closed-form solution and must be numerically computed. The methods for computing these points are given by [26, 28], although various subroutines can be found with a quick Internet search. ChebFun [17] has a particularly good implementation for MATLAB.

There are three commonly used sets of Legendre points. The *Legendre–Gauss* points are given by the roots of the  $P_n(t)$ , the  $n$ th-degree Legendre polynomial. The *Legendre–Gauss–Radau* points are given by the roots of  $P_n(t) + P_{n-1}(t)$ . Finally, the *Legendre–Gauss–Lobatto* points is given by the roots of  $P_{n-1}(t)$  along with the boundary points  $-1$  and  $1$  [24].

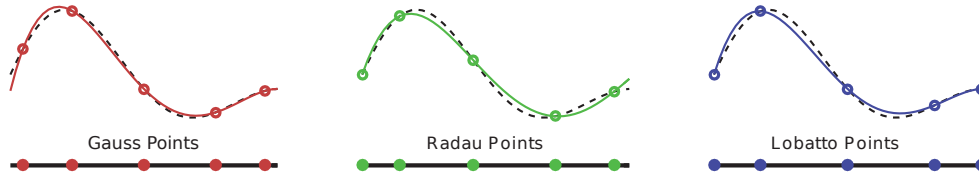
The important distinction between these three sets of points is whether or not the endpoints of the interval are included in a given set of points. Orthogonal collocation schemes can be constructed from any of these sets of points, although they will have different properties [24]. Here we have outlined these points for the Legendre polynomials, but the naming convention (Gauss, Radau, and Lobatto) applies to any orthogonal polynomial. Figure 19 shows an illustration of the Gauss, Radau, and Lobatto points for the Legendre orthogonal polynomials.

Collocation methods whose collocation points include both endpoints of a segment are called *Lobatto* methods. Two popular Lobatto methods are the trapezoidal collocation and Hermite–Simpson collocation methods [6]. A high-order Lobatto method based on Chebyshev orthogonal polynomials is described in [19].

A *Gauss* method is one where the neither endpoint of the segment is a collocation point. A common low-order example would be the implicit midpoint method. A high-order Gauss method based on Legendre orthogonal polynomials is described in [22, 21].

Finally, a *Radau* method is one where a single endpoint of each segment is a collocation point, such as the backward Euler Method. The trajectory optimization software GPOPS [45] uses a high-order Radau method, based on Legendre orthogonal polynomials.

These three types of methods are discussed in more detail in [23, 24] and are illustrated in Figure 19. Garg et al. [24] suggest that high-order Lobatto collocation schemes should be avoided in trajectory optimization, due to poor numerical properties, and that schemes based on Radau and Gauss points should be preferred.



**Fig. 19** Illustration showing the three sets of points that are associated with each orthogonal polynomial. In this figure we have shown the Gauss, Radau, and Lobatto points for the fourth-order Legendre orthogonal polynomials. The dashed line in each figure is the same, and the solid lines show the barycentric interpolant that is defined by that set of collocation points. Notice that the interpolant behaves differently for each set of points.

**D.2. Barycentric Lagrange Interpolation.** The best way to store and evaluate high-order orthogonal polynomials is using barycentric Lagrange interpolation. This works by expressing the value of the function at any point  $f(t)$  using a weighted combination of the function's value ( $f_i = f(t_i)$ ) at the roots of the orthogonal polynomial ( $t_i$ ). The equation for barycentric interpolation is given below, with further details in [4]. Note that this expression is not valid when evaluated at the interpolation points  $t = t_i$ . This is not a problem, since the value of the function at these points is already known to be  $f_i$ :

$$(D.3) \quad f(t) = \frac{\sum_{i=0}^n \frac{v_i}{t - t_i} f_i}{\sum_{i=0}^n \frac{v_i}{t - t_i}}.$$

Thus far, we know all parameters in (D.3) except for the interpolation weights  $v_i$ . These weights are calculated below, using the equation given by [4]:

$$(D.4) \quad v_i = \frac{1}{\prod_{j \neq i} (t_i - t_j)}, \quad i = 0, \dots, n.$$

Interestingly, the barycentric interpolation formula (D.3) will still interpolate the data at points  $f_i$  if the weights  $v_i$  are chosen arbitrarily. The choice of weights given by (D.4) is special in that it defines the unique *polynomial* interpolant, while any other choice of weights will result in interpolation by some rational function [4].

Notice that these weights can be scaled by an arbitrary constant and still produce the correct interpolation in (D.3), as well as the correct differentiation matrix (D.6). For example, ChebFun [17] normalizes the barycentric weights such that the magnitude of the largest weight is 1.

In an orthogonal collocation method, barycentric interpolation would be used to evaluate the solution. It is not used when constructing the nonlinear program.

**D.3. Differentiation Matrix.** Another useful property of orthogonal polynomials is that they are easy to differentiate. Let's define a column vector  $\mathbf{f} = [f_0, f_1, \dots, f_n]^T$  which contains the value of  $f()$  at each interpolation point  $t_i$ . It turns out that we can find some matrix  $\mathcal{D}$  that can be used to compute the derivative of  $f()$  at each interpolation point (D.5):

$$(D.5) \quad \dot{\mathbf{f}} = \mathcal{D}\mathbf{f}.$$

Each element of the differentiation matrix  $\mathcal{D}$  can be computed as shown below, using a formula from [4]:

$$(D.6) \quad \mathcal{D}_{ij} = \begin{cases} \frac{v_j/v_i}{t_i - t_j}, & i \neq j, \\ -\sum_{i \neq j} \mathcal{D}_{ij}, & i = j. \end{cases}$$

We can use the same interpolation weights  $v_i$  for interpolation of this derivative  $\dot{f}(t)$ —we just replace the  $f_i$  terms in (D.3) with  $\dot{f}_i$  to obtain

$$(D.7) \quad \dot{f}(t) = \frac{\sum_{i=0}^n \frac{v_i}{t - t_i} \dot{f}_i}{\sum_{i=0}^n \frac{v_i}{t - t_i}}.$$

**D.4. Quadrature.** Each type of orthogonal polynomial has a corresponding quadrature rule to compute its definite integral. In orthogonal collocation, these quadrature rules are used to evaluate integral constraints and objective functions. The quadrature rule is computed as shown below, and is a linear combination of the function value at each interpolation point ( $t_i$ ):

$$(D.8) \quad \int_{-1}^1 f(\tau) d\tau \approx \sum_{i=0}^n w_i \cdot f_i.$$

Typically these quadrature weights ( $w_i$ ) are computed at the same time as the interpolation points ( $t_i$ ) and weights ( $v_i$ ). Alternatively, the quadrature weights can be determined directly from the interpolation points and weights, although the equations are specific to each type of orthogonal polynomial. For example, the Legendre–Gauss quadrature weights and the Legendre–Gauss–Lobatto weights can be computed as

$$(D.9) \quad w_i = W \frac{v_i^2}{(1 - t_i^2)}, \quad \text{Legendre–Gauss,}$$

$$(D.10) \quad w_i = W v_i^2, \quad \text{Legendre–Gauss–Lobatto.}$$

In both cases the scaling constant  $W$  should be selected such that  $\sum w_i = 2$ . This scaling can be derived by computing the integral of unity  $f_i = 1$ :

$$(D.11) \quad \int_{-1}^1 d\tau = 2 = \sum_{i=0}^n w_i \cdot (1).$$

More details on the calculation of quadrature rules can be found in [58, 20, 32, 65].

**Appendix E. Parameters for Example Problems.** In this section we provide tables for the parameter values that we used when generating the results for both the cart-pole swing-up example problem and the five-link biped example problem.

**E.1. Cart-Pole Swing-Up Parameters.** For the cart-pole swing-up problem we chose parameters for our model to match something that might be seen in a cart-pole in a controls lab demonstration. given in Table 3.

**E.2. Five-Link Biped Parameters.** For the five-link biped walking gait example we chose parameters for our model to match the walking robot RABBIT [66, 13], which are reproduced in Table 4. We also selected a trajectory duration of  $T = 0.7s$  and a step length of  $D = 0.5m$ .

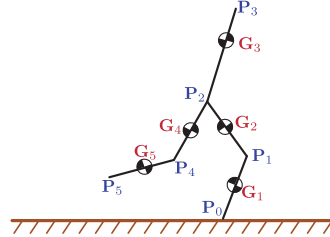


**Table 3** Physical parameters for the cart-pole example.

| Symbol     | Value                 | Name                                    |
|------------|-----------------------|-----------------------------------------|
| $m_1$      | 1.0 kg                | mass of cart                            |
| $m_2$      | 0.3 kg                | mass of pole                            |
| $\ell$     | 0.5 m                 | pole length                             |
| $g$        | 9.81 m/s <sup>2</sup> | gravity acceleration                    |
| $u_{\max}$ | 20 N                  | maximum actuator force                  |
| $d_{\max}$ | 2.0 m                 | extent of the rail that cart travels on |
| $d$        | 1.0 m                 | distance traveled during swing-up       |
| $T$        | 2.0 s                 | duration of swing-up                    |

**Table 4** Physical parameters for the five-link biped model (RABBIT) [13].

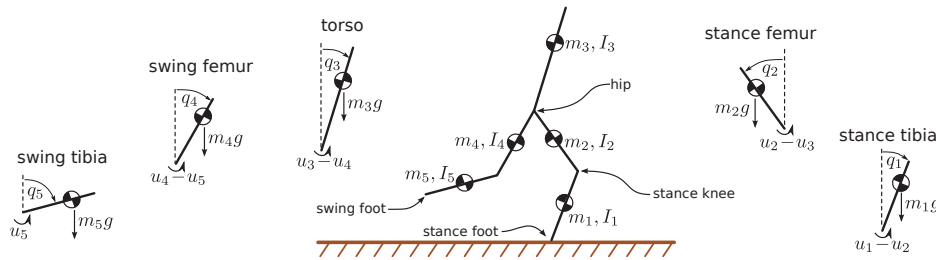
| Symbol           | Value                  | Name                                                  |
|------------------|------------------------|-------------------------------------------------------|
| $m_1, m_5$       | 3.2 kg                 | mass of tibia (lower leg)                             |
| $m_2, m_4$       | 6.8 kg                 | mass of femur (upper leg)                             |
| $m_3$            | 20 kg                  | mass of torso                                         |
| $I_1, I_5$       | 0.93 kg-m <sup>2</sup> | rotational inertia of tibia, about its center of mass |
| $I_2, I_4$       | 1.08 kg-m <sup>2</sup> | rotational inertia of femur, about its center of mass |
| $I_3$            | 2.22 kg-m <sup>2</sup> | rotational inertia of torso, about its center of mass |
| $\ell_1, \ell_5$ | 0.4 m                  | length of tibia                                       |
| $\ell_2, \ell_4$ | 0.4 m                  | length of femur                                       |
| $\ell_3$         | 0.625 m                | length of torso                                       |
| $d_1, d_5$       | 0.128 m                | distance from tibia center of mass to knee            |
| $d_2, d_4$       | 0.163 m                | distance from femur center of mass to hip             |
| $d_3$            | 0.2 m                  | distance from torso center of mass to hip             |

**Fig. 20** Kinematics for the five-link biped model. The illustration shows both joints  $P_i$  and the center of mass of each link  $G_i$ .

**Appendix F. Biped Dynamics.** In this section we will cover some of the more detailed calculations for the five-link biped model of walking, including kinematics, single stance dynamics, heel-strike dynamics, and gradients. We will assume that the reader has a solid understanding of the dynamics of rigid body mechanisms, as well as experience in deriving equations of motion using a symbolic algebra computer package, such as the MATLAB Symbolic Toolbox [38].

**F.1. Kinematics.** Let's start by defining the position vectors that point from the origin  $P_0$  to each joint of the robot,  $P_i$  and the center of mass of each link  $G_i$ , as shown in Figure 20. Each of these position vectors is dependent on the configuration of the robot,  $P_i = P_i(\mathbf{q})$  and  $G_i = G_i(\mathbf{q})$ , where  $\mathbf{q} = [q_1 \ q_2 \ q_3 \ q_4 \ q_5]^T$  is a column vector of absolute link orientations. We will define  $P_0 = \mathbf{0}$ .

There are many ways to compute the position vectors. Here we work from the root joint  $P_0$  outward along the kinematic chain, defining each successive position  $P_i$



**Fig. 21** Dynamics model for the five-link biped model, shown here in single stance. We assume that the dynamics are planar (2D) and modeled as a kinematic chain, with each link assigned a number: 1 = stance tibia, 2 = stance femur, 3 = torso, 4 = swing femur, and 5 = swing tibia. Each joint is connected to its parent by an ideal revolute joint and torque source. Joint torques are given by  $u_i$ , link masses and inertias by  $m_i$  and  $I_i$ , and gravity is  $g$ . The absolute orientation of each link is given by  $q_i$ .

in terms of a previously defined position vector  $P_{i-1}$  and a relative vector in the link frame.

Once the position vectors are defined, we compute velocity and acceleration vectors using the chain rule. The velocities are given below, where  $\dot{\mathbf{q}} = [\dot{q}_1 \ \dot{q}_2 \ \dot{q}_3 \ \dot{q}_4 \ \dot{q}_5]^T$  is the vector of absolute angular rates:

$$(F.1) \quad \dot{P}_i = \left( \frac{\partial P_i}{\partial \mathbf{q}} \right) \dot{\mathbf{q}}, \quad \dot{G}_i = \left( \frac{\partial G_i}{\partial \mathbf{q}} \right) \dot{\mathbf{q}}.$$

The calculation for the acceleration vectors is carried out in a similar fashion, although we need to include the joint rates in the list of partial derivatives. We can do this by defining  $\mathbf{z} = [\mathbf{q} \ \dot{\mathbf{q}}]^T$  and  $\dot{\mathbf{z}} = [\dot{\mathbf{q}} \ \ddot{\mathbf{q}}]^T$ , where  $\ddot{\mathbf{q}} = [\ddot{q}_1 \ \ddot{q}_2 \ \ddot{q}_3 \ \ddot{q}_4 \ \ddot{q}_5]^T$ :

$$(F.2) \quad \ddot{P}_i = \left( \frac{\partial \dot{P}_i}{\partial \mathbf{z}} \right) \dot{\mathbf{z}}, \quad \ddot{G}_i = \left( \frac{\partial \dot{G}_i}{\partial \mathbf{z}} \right) \dot{\mathbf{z}}.$$

Both of these calculations (F.1) and (F.2) can be implemented in MATLAB with the following commands, where all variables are defined to be column vectors:

```
>> dP = Jacobian(P,q)*dq;
>> dG = Jacobian(G,q)*dq;
>> ddP = Jacobian(dP,[q; dq])*[dq; ddq];
>> ddG = Jacobian(dG,[q; dq])*[dq; ddq];
```

**F.2. Single-Stance Dynamics.** In trajectory optimization it is best to use a minimal coordinate formulation of the dynamics: one where there is one equation for each degree of freedom. For this example we will use the absolute angle of each link in the robot for the minimal coordinates, as shown in Figure 21, and compute their accelerations (the equations of motion) using the Newton–Euler equations. Although it is possible to derive these equations by hand, we suggest that you use a computer algebra package for the derivation, such as the MATLAB Symbolic Toolbox [38] or the Python Symbolic Library [56].

The goal of the dynamics calculations is to arrive at a set of equations defining the link accelerations  $\ddot{\mathbf{q}}$  in terms of the link angles  $\mathbf{q}$ , rates  $\dot{\mathbf{q}}$ , and torques  $\mathbf{u} = [u_1 \ u_2 \ u_3 \ u_4 \ u_5]^T$ . Here we will use computer algebra to generate a linear system of equations, which we will then solve numerically at run time for the accelerations  $\ddot{\mathbf{q}}$ . It turns out that this approach is significantly faster (in both run time and derivation

time) than solving for the joint accelerations explicitly:

$$(F.3) \quad \mathcal{M}(q) \cdot \ddot{q} = \mathcal{F}(q, \dot{q}, u).$$

For our five-link biped, there are five linearly independent equations required to construct (F.3), one for each degree of freedom. One way to construct such a system is to write out the equations for angular momentum balance about each successive joint in the robot. Here we will start with the angular momentum balance of the entire robot about the stance foot joint (below). Note that the left side of the equation is a sum over all external torques applied to the system about point  $P_0$ , the stance foot. The right side of the equation gives the time rate of change in the angular momentum of the system about  $P_0$ :

$$(F.4) \quad u_1 + \hat{k} \cdot \sum_{i=1}^5 \left( (G_i - P_0) \times (-m_i g \hat{j}) \right) = \hat{k} \cdot \sum_{i=1}^5 \left( (G_i - P_0) \times (m_i \ddot{G}_i) + \ddot{q}_i I_i \hat{k} \right).$$

The next equation is obtained by simply moving one joint out along the robot, computing the angular momentum balance about the stance knee  $P_1$ :

$$(F.5) \quad u_2 + \hat{k} \cdot \sum_{i=2}^5 \left( (G_i - P_1) \times (-m_i g \hat{j}) \right) = \hat{k} \cdot \sum_{i=2}^5 \left( (G_i - P_1) \times (m_i \ddot{G}_i) + \ddot{q}_i I_i \hat{k} \right).$$

The remaining three equations are given below, following a similar pattern. Notice that the pattern breaks down slightly at the hip joint, because links 3 and 4 are both connected to the hip joint  $P_2$ :

$$(F.6) \quad u_3 + \hat{k} \cdot \sum_{i=3}^5 \left( (G_i - P_2) \times (-m_i g \hat{j}) \right) = \hat{k} \cdot \sum_{i=3}^5 \left( (G_i - P_2) \times (m_i \ddot{G}_i) + \ddot{q}_i I_i \hat{k} \right),$$

$$(F.7) \quad u_4 + \hat{k} \cdot \sum_{i=4}^5 \left( (G_i - P_2) \times (-m_i g \hat{j}) \right) = \hat{k} \cdot \sum_{i=4}^5 \left( (G_i - P_2) \times (m_i \ddot{G}_i) + \ddot{q}_i I_i \hat{k} \right),$$

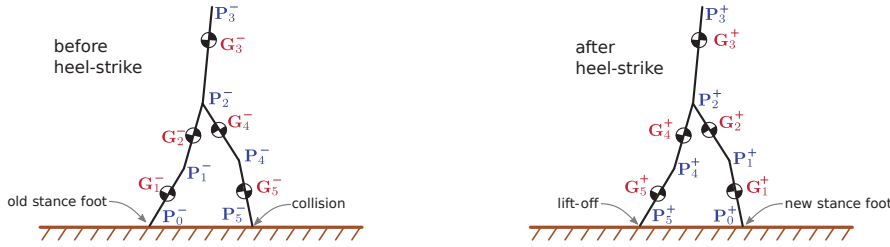
$$(F.8) \quad u_5 + \hat{k} \cdot \sum_{i=5}^5 \left( (G_i - P_4) \times (-m_i g \hat{j}) \right) = \hat{k} \cdot \sum_{i=5}^5 \left( (G_i - P_4) \times (m_i \ddot{G}_i) + \ddot{q}_i I_i \hat{k} \right).$$

**F.3. Heel-Strike Dynamics.** For our biped walking model, we will assume that the biped transitions directly from single stance on one foot to single stance on the other: as soon as the leading foot strikes the ground, the trailing foot leaves the ground. This transition is known as a heel-strike map. We will also assume that this transition occurs instantaneously and that the robot is symmetric.

There are two parts to the heel-strike map. The first is an impulsive collision, which changes the joint velocities throughout the robot, but does not affect the configuration (angles). The second part of the map swaps the swing and stance legs. The leg swap is done to enforce a symmetry in the solution: we want the step taken by the left leg to be identical to that for the right, and for both to be periodic.

Figure 22 shows the biped model immediately before and after the heel-strike map. Notice that the old swing foot  $P_0^-$  has become the new stance foot  $P_5^+$  after the map. Similar renaming has been applied throughout the robot and can be computed using the equation:

$$(F.9) \quad q^+ = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix} q^-.$$



**Fig. 22** Illustration of the kinematics of the five-link biped model both before  $-$  and after  $+$  heel-strike. Note that the points on the robot are relabeled during the collision, reflecting the left-right symmetry of the robot.

Next we derive a linear system that relates the angular rates before and after the collision. Like the single stance dynamics, we will solve this system numerically at run time:

$$(F.10) \quad \mathcal{M}_H(q^-) \cdot \dot{q}^+ = \mathcal{F}_H(q^-, \dot{q}^-).$$

One way to derive this system of equations is to observe that it must conserve angular momentum about the collision point, as well as all joints in the robot. The five equations defining the system are given below. Notice that the left side of each equation is the angular momentum before heel-strike, taken about the swing foot (which is about to become the new stance foot). The right side of each equation is the angular momentum after heel-strike, taken about the stance foot (which was previously the swing foot). Figure 22 shows the naming conventions used throughout these equations. Note that the structure of these equations is somewhat similar to those used for the single stance dynamics:

$$(F.11) \quad \hat{k} \cdot \sum_{i=1}^5 \left( (G_i^- - P_5^-) \times (m_i \dot{G}_i^-) + \dot{q}_i^- I_i \hat{k} \right) = \hat{k} \cdot \sum_{i=1}^5 \left( (G_i^+ - P_0^+) \times (m_i \dot{G}_i^+) + \dot{q}_i^+ I_i \hat{k} \right),$$

$$(F.12) \quad \hat{k} \cdot \sum_{i=1}^4 \left( (G_i^- - P_4^-) \times (m_i \dot{G}_i^-) + \dot{q}_i^- I_i \hat{k} \right) = \hat{k} \cdot \sum_{i=2}^5 \left( (G_i^+ - P_1^+) \times (m_i \dot{G}_i^+) + \dot{q}_i^+ I_i \hat{k} \right),$$

$$(F.13) \quad \hat{k} \cdot \sum_{i=1}^3 \left( (G_i^- - P_2^-) \times (m_i \dot{G}_i^-) + \dot{q}_i^- I_i \hat{k} \right) = \hat{k} \cdot \sum_{i=3}^5 \left( (G_i^+ - P_2^+) \times (m_i \dot{G}_i^+) + \dot{q}_i^+ I_i \hat{k} \right),$$

$$(F.14) \quad \hat{k} \cdot \sum_{i=1}^2 \left( (G_i^- - P_2^-) \times (m_i \dot{G}_i^-) + \dot{q}_i^- I_i \hat{k} \right) = \hat{k} \cdot \sum_{i=4}^5 \left( (G_i^+ - P_2^+) \times (m_i \dot{G}_i^+) + \dot{q}_i^+ I_i \hat{k} \right),$$

$$(F.15) \quad \hat{k} \cdot \sum_{i=1}^1 \left( (G_i^- - P_1^-) \times (m_i \dot{G}_i^-) + \dot{q}_i^- I_i \hat{k} \right) = \hat{k} \cdot \sum_{i=5}^5 \left( (G_i^+ - P_4^+) \times (m_i \dot{G}_i^+) + \dot{q}_i^+ I_i \hat{k} \right).$$

Our final step is to combine (F.9) and (F.10) into the heel-strike map equation, shown below, where  $\mathbf{x}^-$  is the state of the system before heel-strike and  $\mathbf{x}^+$  is the state after heel-strike:

$$(F.16) \quad \mathbf{x}^- = \begin{bmatrix} \mathbf{q}^- \\ \dot{\mathbf{q}}^- \end{bmatrix}, \quad \mathbf{x}^+ = \begin{bmatrix} \mathbf{q}^+ \\ \dot{\mathbf{q}}^+ \end{bmatrix},$$

$$(F.17) \quad \mathbf{x}^+ = \mathbf{f}_H(\mathbf{x}^-).$$

**F.4. Gradients.** For trajectory optimization, it is generally a good idea to use analytic gradients where possible. This means that we will need to calculate the following expressions:

$$(F.18) \quad \frac{\partial \ddot{\mathbf{q}}}{\partial \mathbf{q}}, \quad \frac{\partial \ddot{\mathbf{q}}}{\partial \dot{\mathbf{q}}}, \quad \frac{\partial \ddot{\mathbf{q}}}{\partial \mathbf{u}}, \quad \frac{\partial \dot{\mathbf{q}}^+}{\partial \mathbf{q}^-}, \quad \frac{\partial \dot{\mathbf{q}}^+}{\partial \dot{\mathbf{q}}^-}.$$

Unfortunately, we can't use the `Jacobian()` command in the symbolic software, because we plan to calculate  $\ddot{\mathbf{q}}$  and  $\dot{\mathbf{q}}^+$  by numerically solving a linear system at run time. The solution is to use the symbolic software to compute the gradients of  $\mathcal{M}$ ,  $\mathcal{F}$ ,  $\mathcal{M}_H$ , and  $\mathcal{F}_H$  and then derive an expression for the gradient of  $\ddot{\mathbf{q}}$  and  $\dot{\mathbf{q}}^+$  in terms of these known matrices. We start by deriving the gradient of the matrix inverse operator:

$$(F.19) \quad \mathcal{M}^{-1} \mathcal{M} = \mathcal{I},$$

$$(F.20) \quad \frac{\partial}{\partial q_i} (\mathcal{M}^{-1} \mathcal{M}) = \mathbf{0},$$

$$(F.21) \quad \frac{\partial}{\partial q_i} (\mathcal{M}^{-1}) \mathcal{M} + \mathcal{M}^{-1} \frac{\partial}{\partial q_i} (\mathcal{M}) = \mathbf{0},$$

$$(F.22) \quad \frac{\partial \mathcal{M}^{-1}}{\partial q_i} = -\mathcal{M}^{-1} \frac{\partial \mathcal{M}}{\partial q_i} \mathcal{M}^{-1}.$$

We will now apply (F.22) to compute the gradient of the link accelerations  $\ddot{\mathbf{q}}$  with respect to a single link angle  $q_i$ . This process can then be repeated for the partial derivatives with respect to the remaining joint angles, rates  $\dot{q}_i$ , and torques  $u_i$ . These same calculations (F.25) can be applied to the heel-strike calculations:

$$(F.23) \quad \frac{\partial \ddot{\mathbf{q}}}{\partial q_i} = \frac{\partial}{\partial q_i} (\mathcal{M}^{-1} \mathcal{F}),$$

$$(F.24) \quad \frac{\partial \ddot{\mathbf{q}}}{\partial q_i} = \left( -\mathcal{M}^{-1} \frac{\partial \mathcal{M}}{\partial q_i} \mathcal{M}^{-1} \right) \mathcal{F} + \mathcal{M}^{-1} \left( \frac{\partial \mathcal{F}}{\partial q_i} \right),$$

$$(F.25) \quad \frac{\partial \ddot{\mathbf{q}}}{\partial q_i} = \mathcal{M}^{-1} \left( -\frac{\partial \mathcal{M}}{\partial q_i} \ddot{\mathbf{q}} + \frac{\partial \mathcal{F}}{\partial q_i} \right).$$

#### REFERENCES

- [1] S. AGRAWAL, S. SHEN, AND M. V. D. PANNE, *Diverse motion variations for physics-based character animation*, in Proceedings of the 12th ACM SIGGRAPH/Eurographics Symposium on Computer Animation, SCA '13, ACM, New York, 2013, pp. 37–44, <https://doi.org/10.1145/2485895.2485907>. (Cited on p. 868)
- [2] V. M. BECERRA, *PSOPT Optimal Control Solver User Manual*, 2011, <http://www.psopt.org/>. (Cited on p. 888)

- [3] D. A. BENSON, G. T. HUNTINGTON, T. P. THORVALDSEN, AND A. V. RAO, *Direct trajectory optimization and costate estimation via an orthogonal collocation method*, J. Guidance Control Dynam., 29 (2006), pp. 1435–1440, <https://doi.org/10.2514/1.20478>. (Cited on pp. 859, 862, 893)
- [4] J.-P. BERRUT AND L. N. TREFETHEN, *Barycentric Lagrange interpolation*, SIAM Rev., 46 (2004), pp. 501–517, <https://doi.org/10.1137/S0036144502417715>. (Cited on pp. 886, 893, 895, 896)
- [5] J. T. BETTS, *Survey of numerical methods for trajectory optimization*, J. Guidance Control Dynam., 21 (1998), pp. 193–207. (Cited on pp. 853, 854, 866, 885, 887)
- [6] J. T. BETTS, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, SIAM, Philadelphia, PA, 2010, <https://doi.org/10.1137/1.9780898718577>. (Cited on pp. 855, 858, 859, 860, 861, 862, 865, 866, 867, 868, 880, 885, 887, 888, 894)
- [7] J. T. BETTS, *SOS: Sparse Optimization Suite—User’s Guide*, 2013, <https://www.astos.de/products/sos>. (Cited on p. 888)
- [8] P. A. BHOUNSULE, J. CORTELL, A. GREWAL, B. HENDRIKSEN, J. G. D. KARSEN, C. PAUL, AND A. RUINA, *Low-bandwidth reflex-based control for lower power walking: 65 km on a single battery charge*, Internat. J. Robotics Res., 33 (2014), pp. 1305–1321, <https://doi.org/10.1177/0278364914527485>. (Cited on p. 874)
- [9] P. A. BHOUNSULE, J. CORTELL, A. GREWAL, B. HENDRIKSEN, J. G. D. KARSEN, C. PAUL, AND A. RUINA, *MULTIMEDIA EXTENSION #1. Low-bandwidth reflex-based control for lower power walking: 65 km on a single battery charge*, Internat. J. Robotics Res., 2014, [http://www.ijrr.org/ijrr\\_2014/volume33-issue10/IJR%20527485/Multimedia%20extension%201.pdf](http://www.ijrr.org/ijrr_2014/volume33-issue10/IJR%20527485/Multimedia%20extension%201.pdf), [http://www.ijrr.org/ijrr\\_2014/527485.htm](http://www.ijrr.org/ijrr_2014/527485.htm). (Cited on p. 882)
- [10] L. T. BIEGLER AND V. M. ZAVALA, *Large-scale nonlinear programming using IPOPT: An integrating framework for enterprise-wide dynamic optimization*, Comput. Chem. Engrg., 33 (2009), pp. 575–582, <https://doi.org/10.1016/j.compchemeng.2008.08.006>. (Cited on p. 867)
- [11] A. E. BRYSON AND Y.-C. HO, *Applied Optimal Control*, Taylor & Francis, London, 1975. (Cited on pp. 855, 889, 890)
- [12] E. CATTO, *Box2D User Manual*, 2013, <http://box2d.org/about/>. (Cited on p. 868)
- [13] B. C. CHEVALLEREAU, G. ABBA, Y. Aoustin, F. PLESTAN, E. R. WESTERVELT, C. CANUDAS-DE WIT, AND J. W. GRIZZLE, *RABBIT: A testbed for advanced control theory*, IEEE Control Systems Mag., 23 (2003), pp. 57–79, <https://doi.org/10.1109/MCS.2003.1234651>. (Cited on pp. 873, 896, 897)
- [14] E. COUMANS, *Bullet Physics SDK Manual*, 2015, [www.bulletphysics.org](http://www.bulletphysics.org). (Cited on p. 868)
- [15] C. L. DARBY, D. GARG, AND A. V. RAO, *Costate estimation using multiple-interval pseudospectral methods*, J. Spacecraft Rockets, 48 (2011), pp. 856–866, <https://doi.org/10.2514/1.A32040>, <http://arc.aiaa.org/doi/abs/10.2514/1.A32040>. (Cited on pp. 886, 893)
- [16] C. L. DARBY, W. W. HAGER, AND A. V. RAO, *An hp-adaptive pseudospectral method for solving optimal control problems*, Optim. Control Appl. Methods, 32 (2011), pp. 476–502, <https://doi.org/10.1002/oca.957>. (Cited on pp. 865, 866, 887)
- [17] T. A. DRISCOLL, N. HALE, AND L. N. TREFETHEN, *Chebfun Guide*, 1st ed., Pafnuty Publications, Oxford, 2014. (Cited on pp. 894, 895)
- [18] G. ELNAGAR, M. A. KAZEMI, AND M. RAZZAGHI, *The pseudospectral Legendre method for discretizing optimal control problems*, IEEE Trans. Automat. Control, 40 (1995), pp. 1793–1796. (Cited on p. 893)
- [19] G. N. ELNAGAR AND M. A. KAZEMI, *Pseudospectral Chebyshev optimal control of constrained nonlinear dynamical systems*, Comput. Optim. Appl., 217 (1998), pp. 195–217, <http://www.springerlink.com/index/h0378m3v1g4j5mq1.pdf>. (Cited on pp. 893, 894)
- [20] B. FORNBERG, *A Practical Guide to Pseudospectral Methods*, Cambridge University Press, Cambridge, UK, 1996. (Cited on p. 896)
- [21] C. C. FRANCOLIN, D. A. BENSON, W. W. HAGER, AND A. V. RAO, *Costate estimation in optimal control using integral Gaussian quadrature orthogonal collocation methods*, Optim. Control Appl. Methods, 36 (2015), pp. 381–397. (Cited on pp. 893, 894)
- [22] D. GARG, W. W. HAGER, AND A. V. RAO, *Gauss pseudospectral method for solving infinite-horizon optimal control problems*, in AIAA Guidance, Navigation, and Control Conference, AIAA, 2010, pp. 1–9. (Cited on p. 894)
- [23] D. GARG, M. PATTERSON, AND W. HAGER, *An overview of three pseudospectral methods for the numerical solution of optimal control problems*, Adv. Astronaut. Sci., 135 (2009), pp. 1–17, <http://vdol.mae.ufl.edu/ConferencePublications/unifiedFrameworkAAS.pdf>. (Cited on pp. 893, 894)

- [24] D. GARG, M. PATTERSON, W. W. HAGER, A. V. RAO, D. A. BENSON, AND G. T. HUNTINGTON, *A unified framework for the numerical solution of optimal control problems using pseudospectral methods*, Automatica, 46 (2010), pp. 1843–1851, <https://doi.org/10.1016/j.automatica.2010.06.048>. (Cited on pp. 859, 862, 893, 894)
- [25] P. E. GILL, W. MURRAY, AND M. A. SAUNDERS, *User's Guide for SNOPT Version 7: Software for Large-Scale Nonlinear Programming*, 2006, <http://www.sbsi-sol-optimize.com/manuals/SNOPT%20Manual.pdf>. (Cited on pp. 867, 888)
- [26] G. H. GOLUB AND J. H. WELSCH, *Calculation of Gauss quadrature rules*, Math. Comp., 23 (1968), pp. 221–230, <https://doi.org/10.1090/S0025-5718-69-99647-1>. (Cited on pp. 893, 894)
- [27] J. W. GRIZZLE, J. HURST, B. MORRIS, H. W. PARK, AND K. SREENATH, *MABEL, a new robotic bipedal walker and runner*, in Proceedings of the 2008 American Control Conference, IEEE, 2009, pp. 2030–2036, <https://doi.org/10.1109/ACC.2009.5160550>. (Cited on p. 873)
- [28] N. HALE AND A. TOWNSEND, *Fast and accurate computation of Gauss-Legendre and Gauss-Jacobi quadrature nodes and weights*, SIAM J. Sci. Comput., 35 (2013), pp. A652–A674, <https://doi.org/10.1137/120889873>. (Cited on pp. 893, 894)
- [29] C. R. HARGRAVES AND S. W. PARIS, *Direct trajectory optimization using nonlinear programming and collocation*, AIAA J. Guidance, 10 (1987), pp. 338–342, <https://doi.org/10.2514/3.20223>. (Cited on p. 893)
- [30] A. L. HERMAN AND B. A. CONWAY, *Direct optimization using collocation based on high-order Gauss-Lobatto quadrature rules*, AIAA J. Guidance Control Dynam., 19 (1996), pp. 522–529, <https://doi.org/10.2514/3.21662>. (Cited on p. 893)
- [31] D. H. JACOBSON AND D. Q. MAYNE, *Differential Dynamic Programming*, Elsevier, New York, 1970. (Cited on p. 886)
- [32] G. KLEIN AND J.-P. BERRUT, *Linear barycentric rational quadrature*, BIT, 52 (2012), pp. 407–424, <https://doi.org/10.1007/s10543-011-0357-x>. (Cited on p. 896)
- [33] D. P. LAURIE, *Computation of Gauss-type quadrature formulas*, J. Comput. Appl. Math., 127 (2001), pp. 201–217, [https://doi.org/10.1016/S0377-0427\(00\)00506-9](https://doi.org/10.1016/S0377-0427(00)00506-9). (Cited on p. 893)
- [34] L. LIU, M. V. D. PANNE, AND K. YIN, *Guided learning of control graphs for physics-based characters*, ACM Trans. Graphics, 35 (2016), art. 29, <https://doi.org/10.1145/2893476>. (Cited on p. 868)
- [35] D. G. LUENBERGER AND Y. YE, *Linear and Nonlinear Programming*, 3rd ed., Springer, New York, 2008. (Cited on p. 855)
- [36] Y. MA, F. BORRELLI, B. HENCEY, B. COFFEY, S. BENGEE, AND P. HAVES, *Model predictive control for the operation of building cooling systems*, IEEE Trans. Control Systems Technol., 20 (2012), pp. 796–803, <https://doi.org/10.1109/TCST.2011.2124461>. (Cited on p. 884)
- [37] MATHWORKS, *MATLAB Optimization Toolbox*, 2014, <http://www.mathworks.com/products/optimization/>. (Cited on pp. 867, 876, 882, 888)
- [38] MATHWORKS, *MATLAB Symbolic Toolbox*, 2014, <http://www.mathworks.com/products/symbolic>. (Cited on pp. 897, 898)
- [39] D. MAYNE, *A second-order gradient method for determining optimal trajectories of non-linear discrete-time systems*, Internat. J. Control, 3 (1966), pp. 85–95, <https://doi.org/10.1080/00207176608921369>. (Cited on p. 886)
- [40] I. MORDATCH, E. TODOROV, AND Z. POPOVIĆ, *Discovery of complex behaviors through contact-invariant optimization*, ACM Trans. Graphics, 31 (2012), art. 43, <https://doi.org/10.1145/2185520.2335394>. (Cited on pp. 868, 886)
- [41] D. M. MURRAY AND S. J. YAKOWITZ, *Differential dynamic programming and Newton's method for discrete optimal control problems*, J. Optim. Theory Appl., 43 (1984), pp. 395–414, <https://doi.org/10.1007/BF00934463>. (Cited on p. 886)
- [42] A. NG, *Reinforcement Learning and Control*, Machine Learning, Stanford CS 229 Lecture Notes, Stanford University, 2012, <http://cs229.stanford.edu/notes/cs229-notes12.pdf>. (Cited on p. 884)
- [43] H. W. PARK, K. SREENATH, A. RAMEZANI, AND J. W. GRIZZLE, *Switching control design for accommodating large step-down disturbances in bipedal robot walking*, Proceedings IEEE International Conference on Robotics and Automation, 2012, pp. 45–50, <https://doi.org/10.1109/ICRA.2012.6225056>. (Cited on p. 873)
- [44] S. V. PARTER, *On the Legendre-Gauss-Lobatto Points and Weights*, J. Scientific Computing, 14 (1999), pp. 347–355. (Cited on p. 893)
- [45] M. A. PATTERSON AND A. V. RAO, *GPOPS II: A MATLAB software for solving multiple-phase optimal control problems using hp-adaptive Gaussian quadrature collocation methods and*

- sparse nonlinear programming*, ACM Trans. Math. Software, 39 (2013), art. 1, <http://doi.acm.org/10.1145/2558904>. (Cited on pp. 853, 865, 879, 886, 888, 894)
- [46] X. B. PENG, G. BERSETH, AND M. VAN DE PANNE, *Dynamic terrain traversal skills using reinforcement learning*, ACM Trans. Graph., 34 (2015), art. 80, <http://www.cs.ubc.ca/~van/papers/2015-TOG-terrainRL/index.html>. (Cited on p. 883)
- [47] M. POSA, S. KUIENDERSMA, AND R. TEDRAKE, *Optimization and stabilization of trajectories for constrained dynamical systems*, in Proceedings of the 2016 IEEE International Conference on Robotics and Automation, IEEE, 2016, pp. 1366–1373, <https://doi.org/10.1109/ICRA.2016.7487270>. (Cited on p. 886)
- [48] M. POSA AND R. TEDRAKE, *Direct trajectory optimization of rigid body dynamical systems through contact*, in Algorithmic Foundations of Robotics X, Springer Tracts in Adv. Robotics 86, Springer, Berlin, Heidelberg, 2013, pp. 527–542. (Cited on pp. 868, 886)
- [49] J. PRATT, *Virtual model control: An intuitive approach for bipedal locomotion*, Internat. J. Robotics Res., 20 (2001), pp. 129–143, <https://doi.org/10.1177/02783640122067309>. (Cited on p. 873)
- [50] W. H. PRESS, S. A. TEUKOLSKY, W. T. VETTERLING, AND B. P. FLANNERY, *Numerical Recipes in C*, 2nd ed., Cambridge University Press, Cambridge, UK, 1992. (Cited on p. 862)
- [51] A. RAO, *A survey of numerical methods for optimal control*, Adv. Astronaut. Sci., 135 (2009), pp. 497–528, <https://doi.org/10.1515/jnum-2014-0003>. (Cited on pp. 853, 886, 887)
- [52] I. M. ROSS, *DIDO*, 2001, <http://www.elissarglobal.com/academic/products/>. (Cited on p. 888)
- [53] I. M. ROSS AND F. FAHROO, *Legendre pseudospectral approximations of optimal control problems*, in New Trends in Nonlinear Dynamics and Control and Their Applications, Lect. Notes Control Inf. Sci. 295, Springer, Berlin, 2003, pp. 327–342, [https://doi.org/10.1007/978-3-540-45056-6\\_21](https://doi.org/10.1007/978-3-540-45056-6_21). (Cited on p. 893)
- [54] C. O. SAGLAM AND K. BYL, *Robust policies via meshing for metastable rough terrain walking*, in Proceedings of Robotics: Science and Systems, Berkeley, CA, 2014, <https://doi.org/10.15607/RSS.2014.X.049>. (Cited on p. 873)
- [55] M. SRINIVASAN AND A. RUINA, *Computer optimization of a minimal biped model discovers walking and running*, Nature, 439 (2006), pp. 72–75, <https://doi.org/10.1038/nature04113>. (Cited on pp. 867, 882)
- [56] SYMPY DEVELOPMENT TEAM, *SymPy: Python library for symbolic mathematics*, 2016, <http://www.sympy.org>. (Cited on p. 898)
- [57] T. W. TEE AND L. N. TREFETHEN, *A rational spectral collocation method with adaptively transformed Chebyshev grid points*, SIAM J. Sci. Comput., 28 (2006), pp. 1798–1811, <https://doi.org/10.1137/050641296>. (Cited on p. 893)
- [58] L. N. TREFETHEN, *Approximation Theory and Approximation Practice*, SIAM, Philadelphia, 2013. (Cited on pp. 886, 893, 894, 896)
- [59] V. A. TUCKER, *Energetic cost of locomotion in animals*, Comparative Biochem. Physiol., 34 (1970), pp. 841–846, [https://doi.org/10.1016/0010-406X\(70\)91006-6](https://doi.org/10.1016/0010-406X(70)91006-6). (Cited on p. 874)
- [60] C. D. TWIGG AND D. L. JAMES, *Many-worlds browsing for control of multibody dynamics*, ACM Trans. Graphics, 26 (2007), art. 14, <https://doi.org/10.1145/1276377.1276395>. (Cited on p. 868)
- [61] C. D. TWIGG AND D. L. JAMES, *Backward steps in rigid body simulation*, ACM Trans. Graphics, 27 (2008), art. 25, <https://doi.org/10.1145/1360612.1360624>. (Cited on p. 868)
- [62] J. VLASSENBROECK AND R. V. DOOREN, *A Chebyshev technique for solving nonlinear optimal control problems*, IEEE Trans. Automat. Control, 33 (1988), pp. 333–340, <https://doi.org/10.1109/9.192187>. (Cited on p. 893)
- [63] O. VON STRYK, *User's guide for DIRCOL: A Direct Collocation Method for the Numerical Solution of Optimal Control pPblems*, Lehrstuhl für Höhere Mathematik und Numerische, 1999, <http://www.sim.informatik.tu-darmstadt.de/publ/download/1999-dircol-2.1-guide-short.pdf>. (Cited on pp. 886, 888)
- [64] A. WÄCHTER AND L. T. BIEGLER, *On the implementation of primal-dual interior point filter line search algorithm for large-scale nonlinear programming*, Math. Program., 106 (2006), pp. 25–57, <https://doi.org/10.1007/s10107-004-0559-y>. (Cited on p. 888)
- [65] H. WANG AND S. XIANG, *On the convergence rate of Legendre approximation*, Math. Comp., 81 (2011), pp. 861–877. (Cited on p. 896)
- [66] E. R. WESTERVELT, J. W. GRIZZLE, AND D. E. KODITSCHKE, *Hybrid zero dynamics of planar biped walkers*, IEEE Trans. Automat. Control, 48 (2003), pp. 42–56, <https://doi.org/10.1109/TAC.2002.806653>. (Cited on pp. 873, 875, 896)
- [67] T. YANG, E. R. WESTERVELT, A. SERRANI, AND J. P. SCHMIEDELER, *A framework for the control of stable aperiodic walking in underactuated planar bipeds*, Autonomous Robots, 27 (2009), pp. 277–290, <https://doi.org/10.1007/s10514-009-9126-y>. (Cited on p. 873)



# AN INTRODUCTION TO TRAJECTORY OPTIMIZATION: HOW TO DO YOUR OWN DIRECT COLLOCATION — CORRECTION TO EQN. 4.13

MATTHEW KELLY \*

**1. Introduction.** In the original paper [1] there is an error in Eqn 4.13. This error was caused by incorrectly computing the integral from the previous equation. This is an isolated error: there are no other equations that depend on it and it was implemented correctly in the attached electronic supplement.

**Original Eqn 4.12 (correct, included for context).** Equation 4.12 shows the state interpolation equation for Hermite–Simpson direct collocation. It is expressed here as an integral using the system dynamics at the collocation points. Note that  $\tau = t - t_k$  and  $h_k = t_{k+1} - t_k$ .

$$\mathbf{x}(t) = \int \dot{\mathbf{x}} dt = \int \left[ \mathbf{f}_k + \left( -3\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} - \mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right) + \left( 2\mathbf{f}_k - 4\mathbf{f}_{k+\frac{1}{2}} + 2\mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right)^2 \right] dt$$

**Original Eqn 4.13 (incorrect, should follow from 4.12).** Equation 4.13 shows the state interpolation equation for Hermite Simpson collocation, and is derived by solving the integral from Eqn. 4.12. In this case the segment duration  $h_k$  was incorrectly treated as a time-varying term, allowing it to be grouped with the time  $\tau$ .

$$\mathbf{x}(t) = \mathbf{x}_k + \mathbf{f}_k \left( \frac{\tau}{h_k} \right) + \frac{1}{2} \left( -3\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} - \mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right)^2 + \frac{1}{3} \left( 2\mathbf{f}_k - 4\mathbf{f}_{k+\frac{1}{2}} + 2\mathbf{f}_{k+1} \right) \left( \frac{\tau}{h_k} \right)^3$$

**Updated Eqn 4.13 (correct, follows from 4.12).** In this version of the equation the segment duration  $h_k$  is treated as constant (with respect to time), and gets pulled out of the integral, rather than being grouped with with time  $\tau$ . This equations is correct, and also matches the interpolation equation found in the OptimTraj software (included in the electronic supplement).

$$\mathbf{x}(t) = \mathbf{x}_k + \left( \mathbf{f}_k \right) \tau + \frac{1}{2h_k} \left( -3\mathbf{f}_k + 4\mathbf{f}_{k+\frac{1}{2}} - \mathbf{f}_{k+1} \right) \tau^2 + \frac{1}{3h_k^2} \left( 2\mathbf{f}_k - 4\mathbf{f}_{k+\frac{1}{2}} + 2\mathbf{f}_{k+1} \right) \tau^3$$

## REFERENCES

- [1] M. P. KELLY, *An introduction to trajectory optimization: how to do your own direct collocation*, SIAM Review, 59 (2017), pp. 849–904.

---

\*Cornell University, Ithaca, NY. (mpk72@cornell.edu). Questions, comments, or corrections to this document may be directed to that email address.